



POLITECNICO

MILANO 1863

054323 – Software Engineering 2 2025
DD

Design Document

Best Bike Paths

Juan Sebastian Sarmiento Pulido- 11153648
Pedro Augusto Villarinho dos Santos - 11108811
Esteban Lopez Barreto - 11154567

Wednesday 7th January, 2026

Version 3.0

Contents

1	INTRODUCTION	2
1.1	Purpose	2
1.2	Scope	2
1.3	Definitions, Acronyms, Abbreviations	2
1.3.1	Definitions	2
1.3.2	Acronyms	3
1.3.3	Abbreviations	3
1.4	Revision History	3
1.5	Reference Documents	4
1.6	Document Structure	4
2	ARCHITECTURAL DESIGN	5
2.1	Overview: High-level components and their interaction	5
2.2	Component view	8
2.3	Deployment view	14
2.4	Runtime view	17
2.5	Component interfaces	30
2.6	Selected architectural styles and patterns	38
2.7	Other design decisions	41
3	USER INTERFACE DESIGN	42
4	REQUIREMENTS TRACEABILITY	51
5	IMPLEMENTATION , INTEGRATION & TESTING	56
5.1	Overview	56
5.2	Implementation Plan	56
5.3	Component Integration and Testing	57
5.4	System Testing	58
5.4.1	Functional System Testing	59
5.4.2	Performance Testing	59
5.4.3	End-to-End Testing	60
6	EFFORT SPENT	60
7	REFERENCES	60

1 INTRODUCTION

1.1 Purpose

Cyclists often face the challenge of finding a safe and well-maintained route for their journeys. Information about bike paths is often incomplete and scattered through multiple sources such as online forums and even word-of-mouth, making it difficult for riders to plan their trips and avoid obstacles such as potholes or poorly maintained roads. In addition to that, cyclists also lack accessible tools to track their own performance while doing these trips. The purpose of BBP is to address all these issues by providing an integrated platform where users can record, analyze, and share their cycling experience. Through user driven data collection, BBP should be able to collect detailed information on roads, road conditions, and performance metrics, ensuring a community-driven system that promotes safety, collaboration, and smarter urban cycling.

1.2 Scope

The aim of this project is to support cyclists in discovering, recording, and sharing bike paths, giving them tools for the creation of highly detailed cycling routes as well as a platform where user's submissions help keep an updated inventory of them. Users must provide information about their rides either manually or automatically, in the latter case, using the automatic path recognition mechanisms offered by the platform, which identify all kinds of information, which must then be confirmed by the user .

The platform will facilitate the search, visualization and dissemination of routes among different users, through the platform selection of the optimal routes, based on the published routes metrics and user confirmation and new experiences, getting a final result that will adapt to the changes in the environment noticed by the users.

The platform will be served on a robust architecture based mainly on a hybrid architecture, mixing microservices with a section of event-driven architecture. This architecture will be supported by different design decisions, such as the inclusion of a service discovery or an inverse-proxy, to provide the architecture that matches the requirements of the stakeholders and provides a robust base to the user to perform operations in.

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Definitions

- **Bike Paths:** A route recorded and defined by a user while cycling, intended to follow streets or trails where a proper bike track exists, or where motor traffic is minimal and speed limits are compatible with typical cycling speeds.
- **Status:** A categorical label describing the overall condition of a Bike Path (Optimal, Medium, Sufficient, Requires maintenance).

- Obstacles: Physical elements along a bike path that may affect cycling, such as potholes, bumps, debris, construction sites or others.
- Statistics: Metrics associated with a recorded Bike Path and shown in the user's records; at minimum these include total distance and duration, and the system may include additional computed metrics derived from the recorded Bike Path when available such as average speed, maximum speed, elevation gain and weather conditions.
- Path Score: A numerical value used to rank Bike Paths for search results, reflecting route condition and efficiency based on available updates.
- Segment: A contiguous portion of a Bike Path associated with a specific street (or street name).

1.3.2 Acronyms

- BBP: Best Bike Path

1.3.3 Abbreviations

- UC: Use Case

1.4 Revision History

- Version 0.1: Setup
 - Created first layout.
- Version 1.0: First release
 - Section 1 added.
 - Section 2 added.
 - Section 3 added.
- Version 2.0: Second release
 - Section 1 added.
 - Section 2 added.
 - Section 3 completed.
 - Section 4 completed.
 - Effort spent section added.
 - Reference Section added.
- Version 3.0: Last release
 - Section 1 completed.

- Section 2 completed.
- Section 3 completed.
- Section 4 completed.
- Effort Spent Section completed.
- Reference Section completed.

1.5 Reference Documents

The assignment for this document and all the information included refer to the following documentation:

- The specification for the 2025/26 Requirement Engineering and Design Project for the Software Engineering II course.
- The slides on the webeep page of the Software Engineering II course.

1.6 Document Structure

1. Introduction: it aims to give a brief description of the project. In particular it's focused on the reasons and the goals that are going to be achieved with its development, as previously explained in the RASD.
2. Architectural Design: : This part of the document describes at different levels of granularity how the architecture is designed and how it is intended to be deployed and working in a production environment.
3. User Interface Design: This section shows the wire frame that constitutes the platform UI
4. Requirements Traceability: in this section it is explained how the goals defined in the RASD document are satisfied by the interaction between the requirements and the component described in the architectural design section.
5. Implementation, Integration and Test Plan: this section provides a description of the plan to implement subsystems and components, likewise, a description of the plan for the integration of the components and testing of the integration.
6. Effort Spent: shows the effort spent to write this document by each participant
7. Bibliography: it contains the references to any documents and software used to write this artifact and to develop the diagrams and different figures exposed in it.

2 ARCHITECTURAL DESIGN

2.1 Overview: High-level components and their interaction

The BBP platform uses a hybrid architecture that combines microservices with message brokers to create a system that can handle both regular user requests and real-time sensor data streaming, with WebSockets being used as the technology that makes the real-time communication possible

Presentation Layer

The presentation tier includes a Desktop Web App and a Mobile Web App that users interact with. The Desktop Web App handles normal operations like searching for paths and viewing routes using standard HTTP requests. The Mobile Web App does the same thing but also establishes WebSocket connections to continuously send sensor data to the server in real-time, which powers the automatic path recording feature.

Application Layer

The application tier sits between the presentation layer and the data layer. It includes the API Gateway. When a request comes in from either web application, the API Gateway receives it, checks if the user is authorized to perform that action, and then directs it to the appropriate backend service. The API Gateway also maintains the WebSocket connections with the Mobile Web App, allowing it to continuously receive sensor data streams. The Load Balancer works with multiple copies of the API Gateway to distribute incoming requests across them, ensuring that no single gateway instance gets overwhelmed.

The backend is composed of several independent microservices, each following the single responsibility principle by handling a specific functionality.

The Automatic Path Recording Service is a key component that directly receives the continuous stream of sensor data coming from the Mobile Web App through the WebSocket connection. Once it receives this sensor data, the Automatic Path Recording Service publishes it to a message broker, which then sends this data to other services that need it. The Obstacle Detection Service subscribes to this sensor data stream and uses it to detect obstacles along the path. The Statistics Service also subscribes to the sensor data to analyze metrics like velocity, distance and other data. Additionally, the Path Construction Service uses the sensor data segments to construct actual bike paths by piecing together the recorded route segments into complete paths that can be stored and reused as Bike Paths.

The Authentication Service manages user login and authorization, the Search Service helps users find paths and routes efficiently while also scoring them based on the users' submissions, and the Path Service handles both the creation of manually entered paths and the reception of automatically recorded paths from the Automatic Path Recording Service. These microservices communicate with each other using REST APIs for direct requests between services.

Data Layer

Data is stored in three separate databases. PathDB stores both automatically recorded paths and manually created paths, maintaining all path information and route data. SearchDB stores indexed path data and scoring information used for efficient path searching and retrieval. UsersDB stores user credentials and profile information. All the databases are replicated across multiple servers so that if one server fails, the system continues working.

External Services

The system also integrates with external services. The Maps Provider supplies geographic and mapping data needed for route computation. The Weather Service Provider delivers real-time weather information that can influence path recommendations. The Mobile Sensor Manager and Web GPS Receiver collect location and sensor telemetry from user mobile devices, with this data flowing to the backend through the WebSocket connection established by the Mobile Web App.

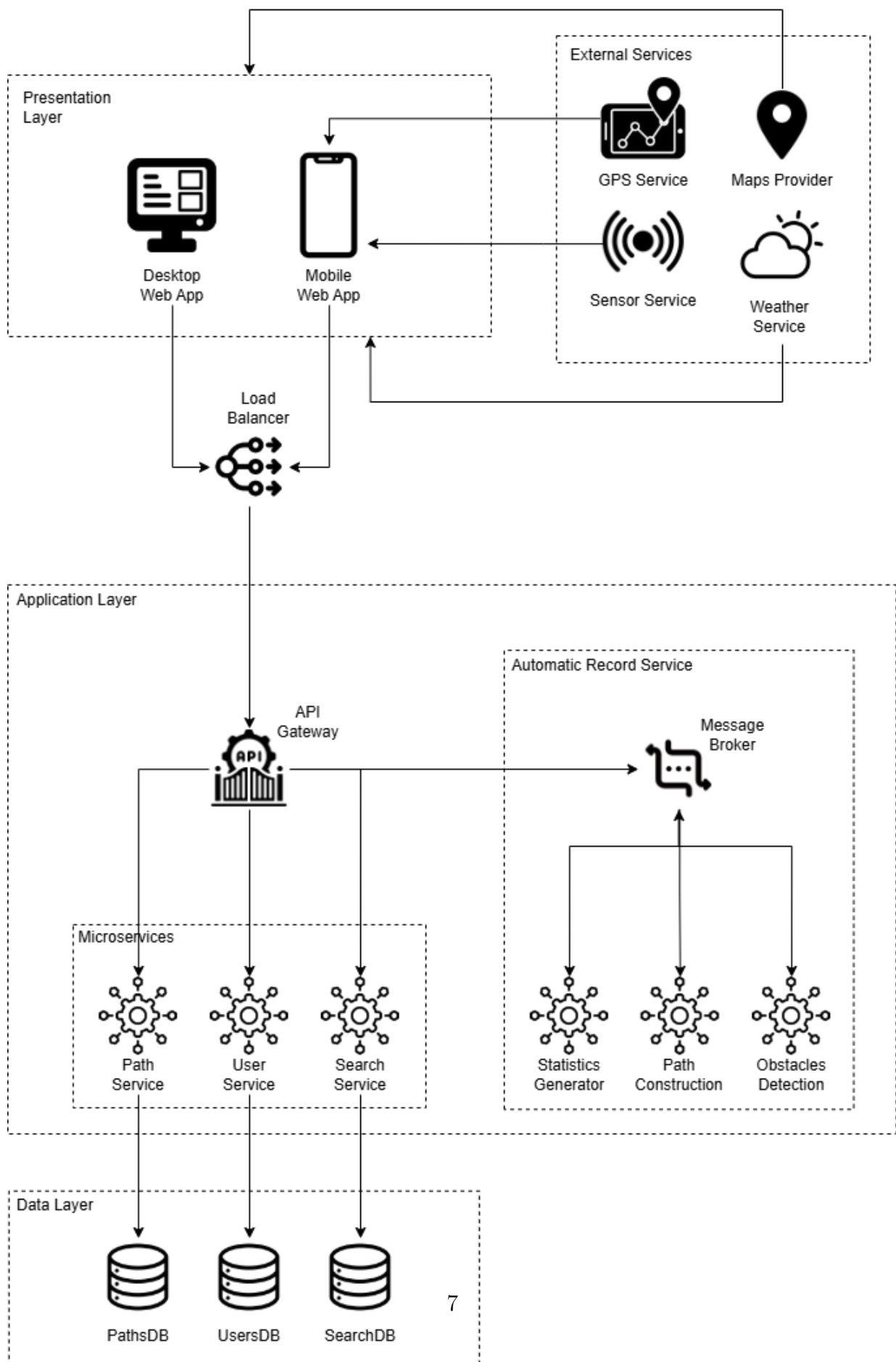


Figure 1: Informal View.

2.2 Component view

The BBP system is composed of various components, each responsible for specific functionalities. These components interact with one another to deliver path recording, storage, analysis, and discovery capabilities. Additionally, the system integrates external services to support these core functionalities.

Presentation Layer

- **Desktop Web App**

This is the component through which desktop users directly interact with the BBP system. It provides a user interface for searching paths, viewing route information, and performing standard operations. The application sends HTTP requests to the API Gateway, which routes them to appropriate backend services. The Desktop Web App is optimized for larger screens and can handle the manual recording of Bike Paths.

- **Mobile Web App**

This is the component through which mobile users interact with the BBP system. It provides the user interface for searching paths and viewing routes, same as the Desktop Web App. The Mobile Web App enables the Automatic Path recording feature establishing WebSocket connections with the API Gateway to stream real-time sensor data from the user's mobile device. The Mobile Web App is also optimized for touch interactions and mobile screens.

Infrastructure Components

- **API Gateway**

This component serves as the central entry point for all client requests from the presentation layer. It receives HTTP requests from both the Desktop Web App and Mobile Web App, and manages WebSocket connections for real-time sensor data streaming from the Mobile Web App. The API Gateway handles request routing to the backend services performing authentication through the Authentication Service. Multiple copies of the API Gateway are deployed to handle incoming traffic at scale.

- **Load Balancer**

This component distributes incoming requests across multiple copies of the API Gateway to ensure no single gateway instance becomes a bottleneck and get overwhelmed. It monitors the health of gateway instances and directs traffic only to available instances. The Load Balancer enables horizontal scaling by allowing additional API Gateway copies to be added as demand increases.

- **Message Broker**

This component enables asynchronous communication between the Automatic Path recording service and its sub components. The Automatic Path Recording Service publishes processed sensor data to the message broker, and other services such as the Path Construction Service, Obstacle Detection Service, and Statistics Service subscribe to this data stream. This way, services are decoupled from each other and can process and send their corresponding data without direct synchronous dependencies.

Application Layer

• Automatic Recording Service

This component receives and processes the continuous stream of sensor data coming from the Mobile Web App through the WebSocket connection. It collects GPS coordinates and sensor metrics from user devices and validates the incoming data. The service publishes raw sensor data to the message broker, where other services consume it to perform their specialized analysis. The service manages the complete recording lifecycle by collecting processed data from the Path Construction Service, Obstacle Detection Service, and Statistics Service through the message broker. It aggregates the processed data from these services to construct a complete bike path that includes path coordinates, detected obstacles, and statistical metadata. Once the recording session is complete, the Automatic Path Recording Service sends the constructed bike path to the Path Service, which stores it in the database.

• Path Construction Service

This component subscribes to the sensor data stream published by the Automatic Path Recording Service through the message broker. It constructs actual bike paths by analyzing and processing the GPS sensor data segments. The service pieces together recorded route segments into complete paths that represent actual bike paths traveled by users. The constructed path is published back to the message broker for the Automatic Path Recording Service to aggregate. This service coordinates with the Maps Provider to ensure that constructed paths are geographically valid and properly mapped.

• Obstacle Detection Service

This component subscribes to the sensor data stream published on the message broker by the Automatic Path Recording Service. It continuously analyzes incoming sensor data to identify potential obstacles or hazards by detecting movement anomalies. The service identifies obstacles and includes them in the processed data. The detected obstacles are published back to the message broker for the Automatic Path Recording Service to include in the final bike path. The obstacles are shown at the end of the recording session.

• Statistics Service

This component subscribes to the sensor data stream published on the message broker to analyze usage metrics and behavioral patterns. It processes the incoming sensor data to calculate statistics such as max velocity, distance traveled, elevation changes and total time. The calculated statistics are published back to the message broker for the Automatic Path Recording Service to include in the final bike path.

- **Path Service**

This component manages the storage and retrieval of all path information within the system. It handles two types of paths: automatically recorded paths that are constructed by the Automatic Path Recording Service, and manually created paths that users enter themselves. The service provides path management and also constructs composite paths that are not listed in the search by using path segments from the database and assigning them to corresponding origins and destinations requested. It exposes APIs that allow other services and user-facing applications to access and manage path data. The Path Service maintains its data in the PathDB database, where all path records, path metadata, and path history are stored.

- **Search Service**

This component helps users discover and find paths and routes efficiently within the system. It maintains an index of all available paths and implements searching functionality. The service calculates rating scores for bike paths based on user submissions and prioritizes those who are frequently updated and valued. The Search Service uses the SearchDB database to store indexed path data, search indices, and scoring information.

- **Authentication Service**

This component manages all user authentication and authorization processes within the BBP platform. It verifies user credentials and manages user sessions. The service stores and retrieves user credentials, authentication tokens, and user profile information from the UsersDB database.

Data Layer

- **PathDB**

This database stores all path-related information within the BBP system. It maintains records of both automatically recorded paths and manually created paths, including complete path coordinates, route segments, obstacles and path metadata.

- **SearchDB**

This database stores indexed paths and their corresponding rating scores submitted by users.

- **UsersDB**

This database stores all user-related information including user credentials and authentication tokens.

External Services

- **Maps Provider**

This service supplies geographic and mapping data to the BBP system. The Maps Provider provides mapping visualization for computed routes and recorded paths in the user-facing applications and returns geographic coordinates and map information needed for path computation and display.

- **Weather Service Provider**

This service delivers real-time weather information to the BBP system. The Statistics Service uses weather data to enrich the statistical information collected during automatic path recording, providing context about weather conditions during path recording sessions.

- **Mobile Sensor Manager**

This external component represents the native sensor capabilities available on user mobile devices, including GPS, accelerometers, and gyroscopes. The Mobile Web App accesses these device sensors to gather location and motion data and transmit it through the WebSocket connection to the Automatic Path Recording Service.

- **Web GPS Receiver**

This external component represents the GPS service available on user mobile devices. The Mobile Web App accesses the device's GPS receiver to obtain accurate positioning information. It works with the Mobile Sensor Manager to ensure reliable GPS coordinates for path recording.

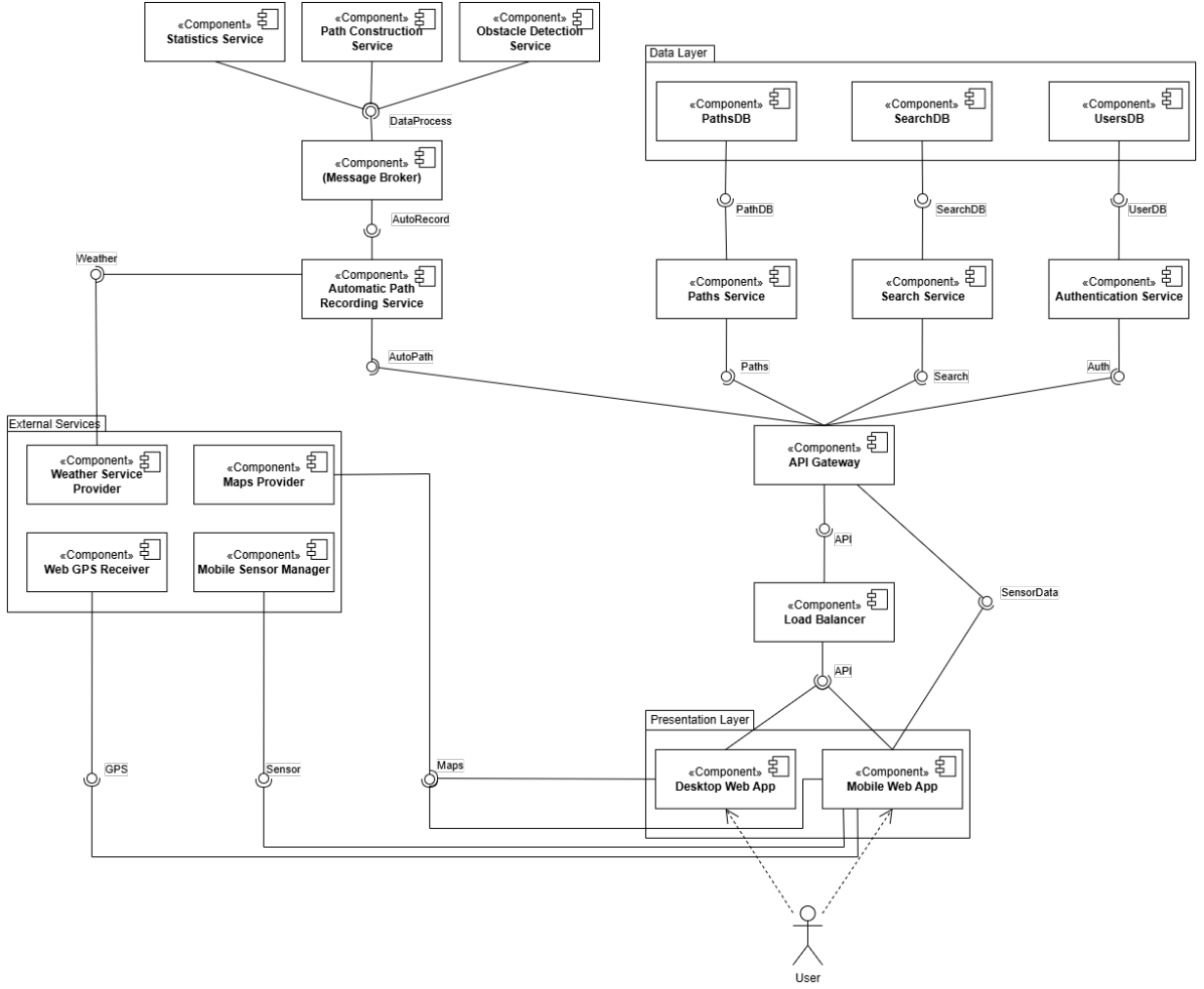


Figure 2: C&C Diagram.

After presenting the system architecture through the Component Diagram, the next step is to introduce the Class Diagram to describe the internal structure of the component, specifically the microservices, where the platform’s core data is manipulated and persisted.

The class diagram provides a more detailed, design-level representation of the system. It focuses on the main classes that directly manage and transform the data produced and consumed by the application, such as trip information, detected obstacles, street/path composition, and performance metrics. The organization of these classes is aligned with the domain class model presented in the RASD, but it also introduces relevant differences. While the RASD class diagram captures the conceptual domain, the diagram presented here refines it towards implementation by detailing the data structures managed and by highlighting the most significant operations required to support key functionalities. For readability, routine methods (getters/setters) are omitted, as they are implicit in the definition of data classes.

To maintain traceability between UML views, the class diagram is organized by mi-

crosservice. This makes explicit which classes belong to each service and clarifies how the overall data model is distributed across the microservice-based architecture.

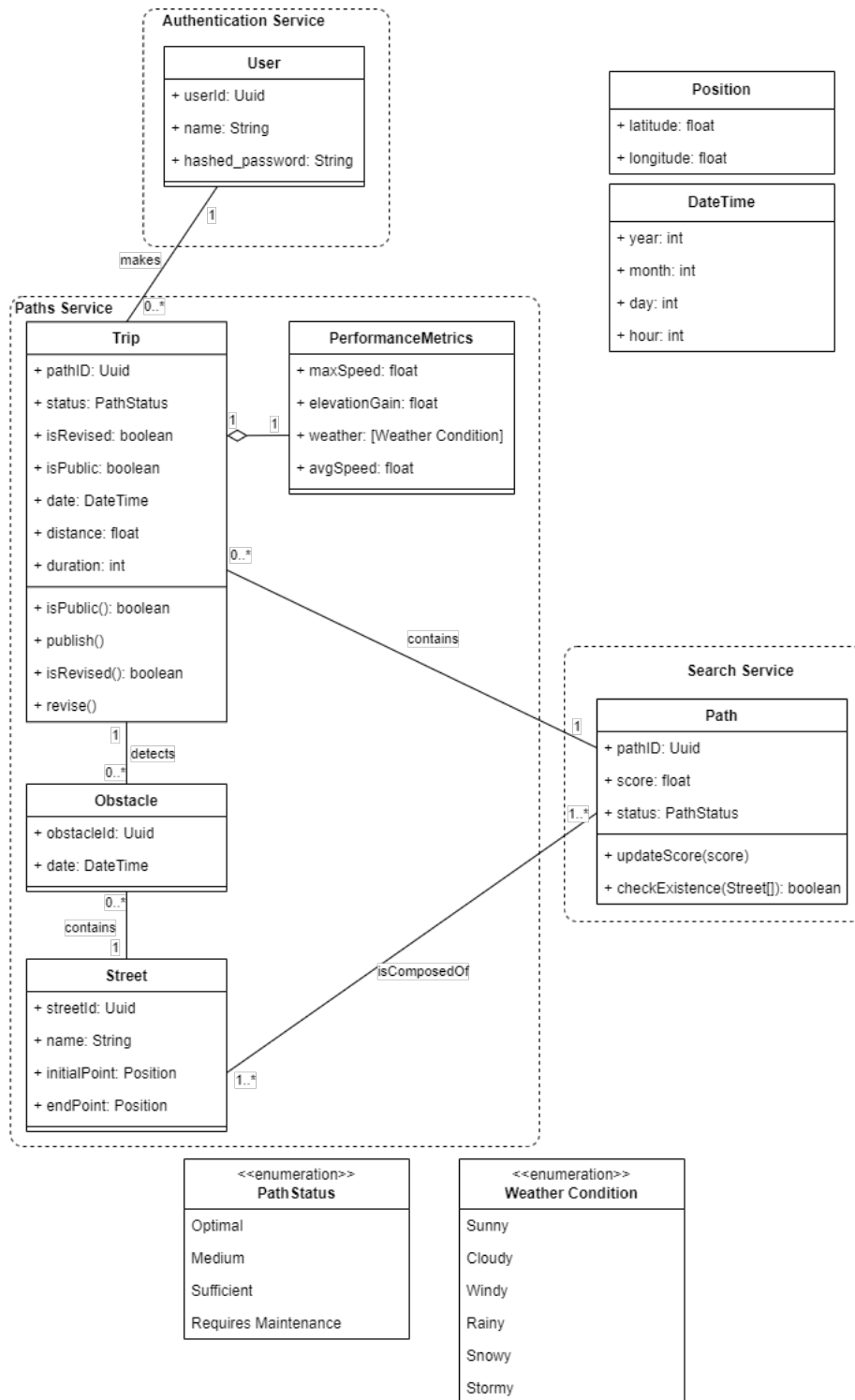
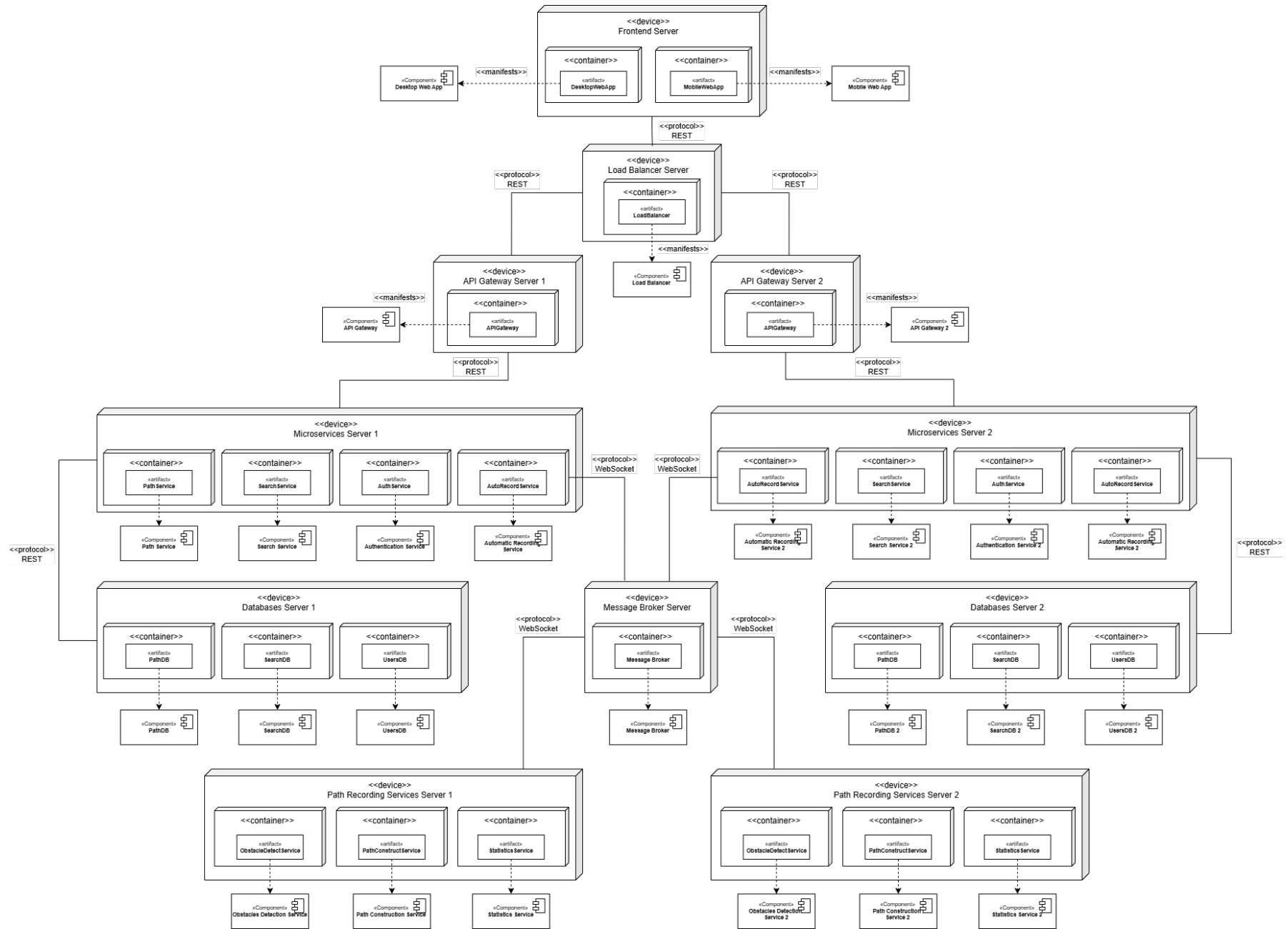


Figure 3: Class Diagram.

2.3 Deployment view

This section describes the deployment view of the BBP system. The system employs a scalable and distributed architecture designed for high availability, fault tolerance, and horizontal scalability.

This deployment has been structured as a horizontally scalable system where almost every server and service is replicated. The diagram presents two instances for each tier (two API gateways, two microservices servers, two database servers, two path recording servers) as a logical representation; in a production environment, the number of replicas can vary and scale higher depending on load and capacity requirements.



Frontend server

The Frontend Server works as the entry point for all users interacting with the BBP system. This device is responsible for hosting and delivering the user-facing client applications that enable users to access the platform. The server contains the Desktop and Mobile version of the app.

Load balancer server

The load balancer server is introduced as the single entry point into the backend infrastructure. It takes client requests and distributes them across the API gateway servers. Although the diagram shows one load balancer node, in production this would typically be a pair or a cluster to prevent the load balancer itself from becoming a single point of failure.

API Gateway server

Each API Gateway Server is responsible for accepting incoming REST requests from clients and determining the appropriate microservice to handle each request.

Each server runs identical containerized gateway components and maintains no local state, allowing them to be completely interchangeable. If one gateway server fails or is taken offline for maintenance, the load balancer seamlessly routes all traffic to the remaining instance without any interruption to ongoing requests.

Microservice server

The Microservice Server makes for the business logic tier of the BBP system. These servers receive routed requests from the API Gateway servers via REST calls, process the requests by accessing databases or the message broker, Services on these servers can communicate with each other through REST APIs for synchronous request-response patterns or through the message broker for asynchronous event-driven communication that does not require immediate responses. and return responses to the gateways. The replication of the servers ensures that if one server experiences issues or needs maintenance, the other server continues processing requests without service interruption.

Database server

The database servers are explicitly drawn as "Database Server 1" and "Database Server 2" to indicate redundancy, but functionally they are replicas of a single logical database. One node usually acts as the primary (or master) for writes while the others act as followers (or slaves), receiving updates through replication logs. Database Server 2 does not introduce a new, independent data model, it is just a replication of the main database to provide failover protection and to serve read traffic. The databases on these servers store different types of data sent from the microservices server in each of the corresponding databases.

Message broker server

The Message Broker Server hosts the message broker component that serves as the central hub for asynchronous communication between all specialized path recording services and the Automatic Recording Service hosted in the microservices server.

Path recording services server

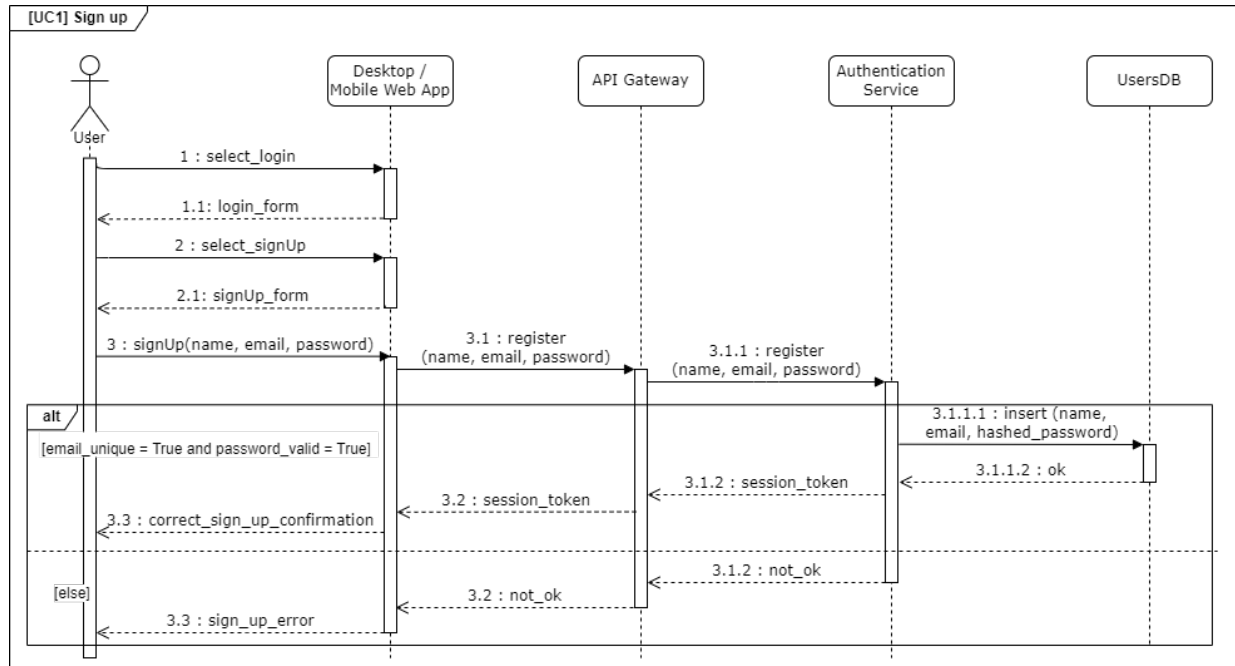
The Path Recording Services Servers host specialized microservices focused on processing sensor data and constructing path information. These services are specialized because they receive continuous streams of sensor data from the Automatic Path Recording Service through the message broker server and must perform computationally intensive analysis in real-time.

2.4 Runtime view

In this section, we describe how the components presented in the Component View interact with each other (through their interfaces) and with the user to achieve the use cases defined in the RASD document: Sign up, Log in, Record path manually, Record path automatically, Revise path obstacles, Visualize paths, Visualize path information, Publish path, Search path, Get alternative path, and Review path.

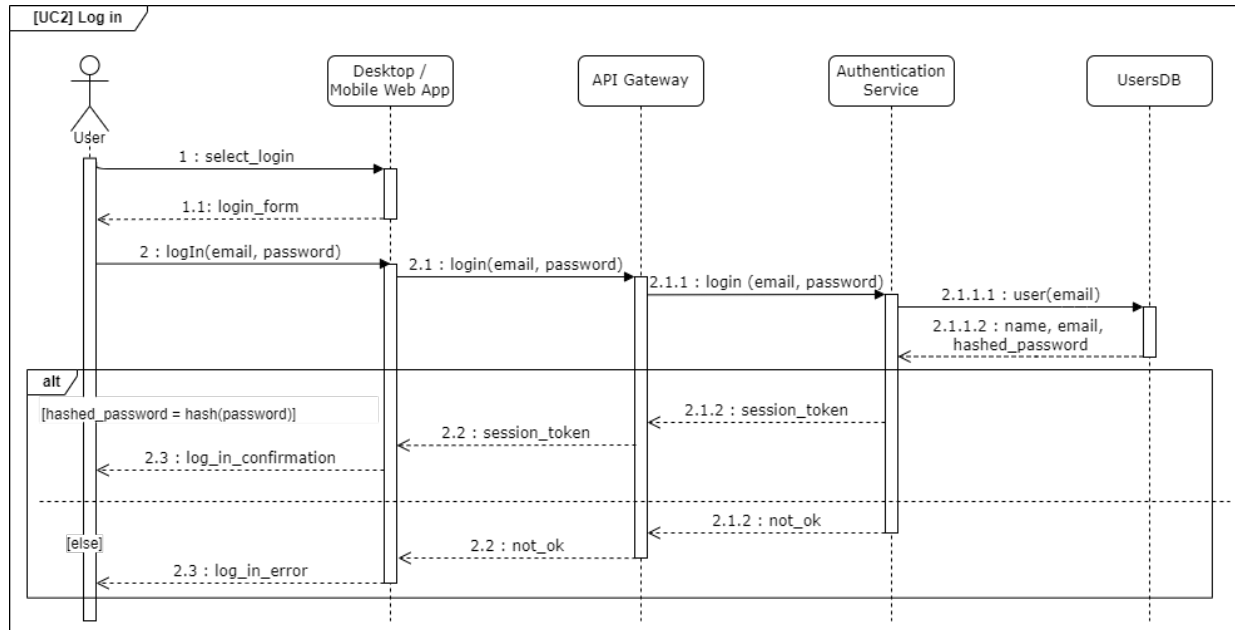
It is important to note that, across all sequence diagrams, the Load Balancer is omitted. Its role is limited to receiving client requests and distributing them among replicas of the API Gateway; therefore, the interfaces involved and the interaction sequences remain unchanged.

[UC1] Sign up



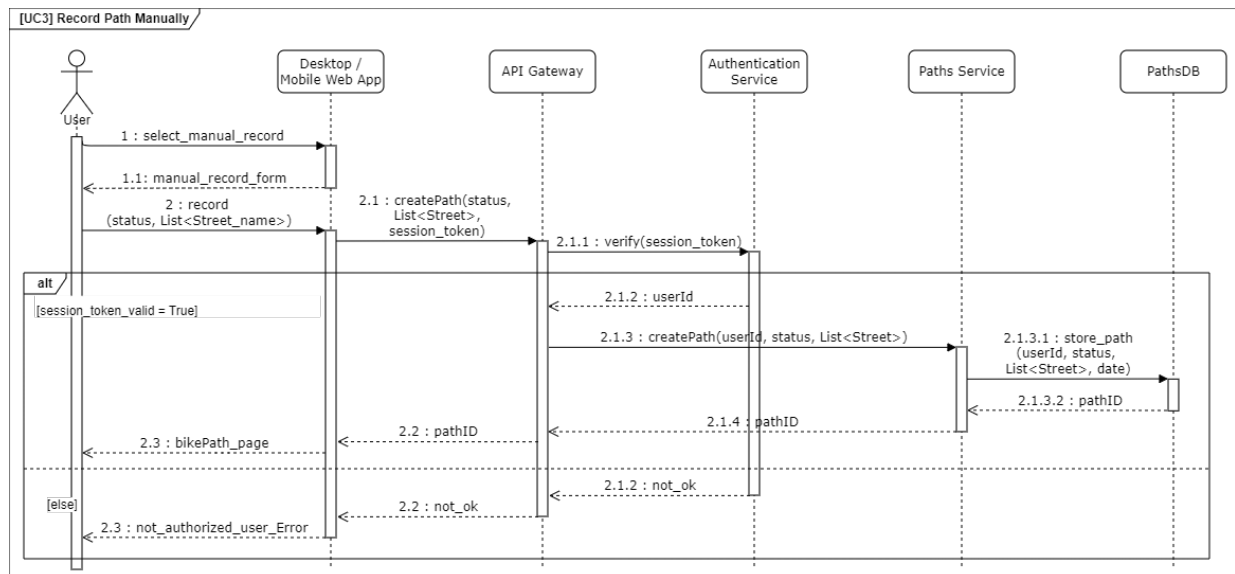
The sequence diagram for the Sign Up use case begins when the user requests the registration form (Figure 7), which is available as a subpage of the Login page (Figure 6). After the user completes and submits the form, the **client application** (either desktop or mobile) sends a registration request to the **API**. The **API** then invokes the **Authentication Service**, which validates the provided data and, if successful, creates and stores the new user account (With the hashed password) for future logins. Finally, the system returns a *session_token*, which will be included in subsequent requests to confirm that the user is authenticated.

[[UC2] Log in



The sequence diagram for the Log In use case begins when the user requests the login form (Figure 6), which is available on the Login page (accessible via a button on the home page - Figure 4). After the user completes and submits the form, the **client application** (either desktop or mobile) sends a login request to the **API**. The **API** then invokes the **Authentication Service**, which validates the credentials provided against the registered user information and, if successful, returns the *session_token*, which is included in subsequent requests to confirm that the user is authenticated.

[UC3] Record path manually

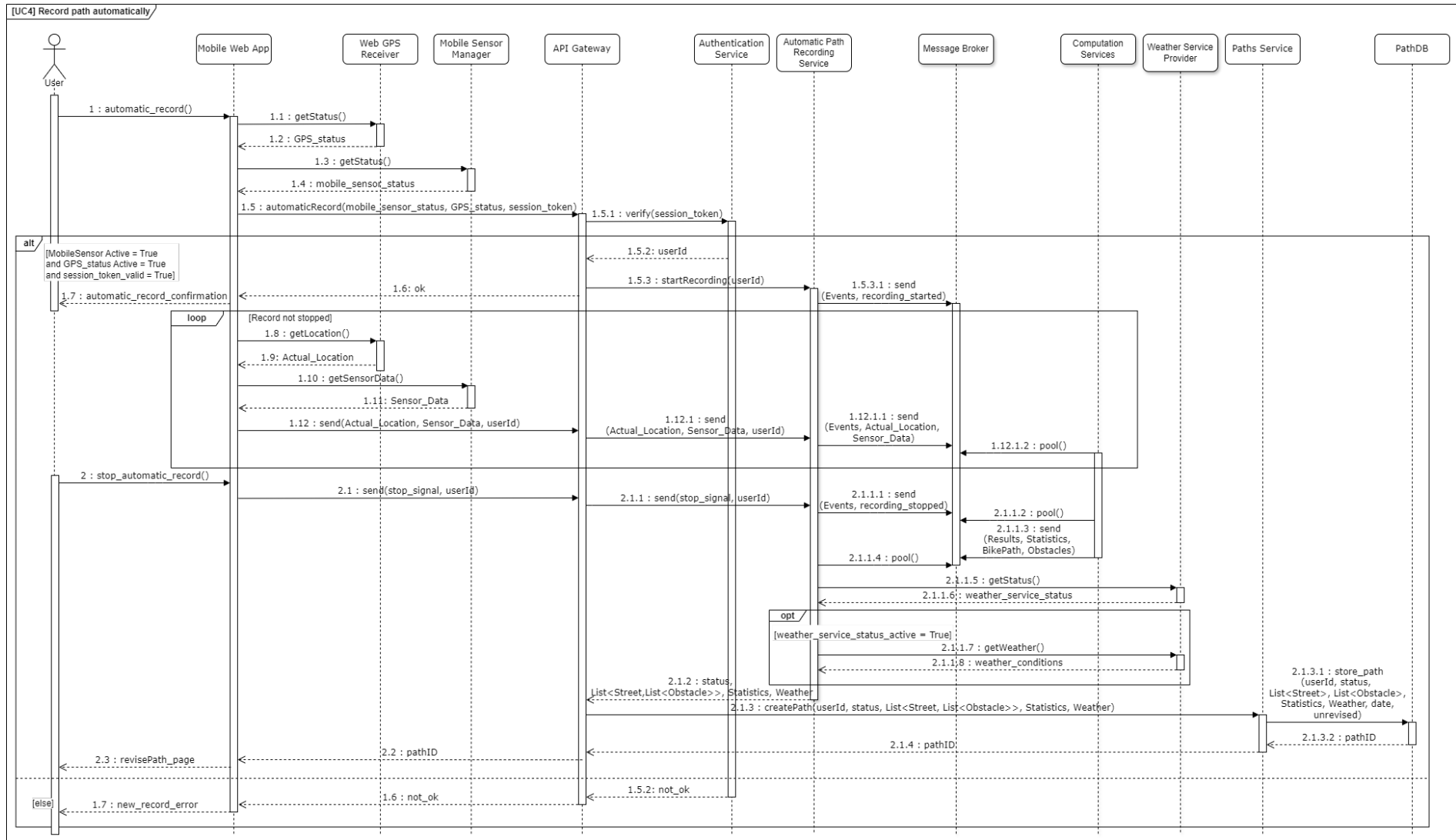


The sequence diagram for the Record path manually use case begins when the user re-

requests the New Record form, which is available on the "Manual Record" page (Figure 16) (accessible via the navbar by selecting "Record" and then "Manual" in the corresponding option - Figure 15). After the user completes and submits the form, the **client application** (either desktop or mobile) sends a path creation request to the **API**.

The API first invokes the **Authentication Service** to verify that the user is authenticated, since this functionality is available only to registered users of the BBP platform. If authentication is successful, the **API** delegates the creation process to the **Paths Service**, which stores the submitted data and returns a *PathID*, which allows the client to be redirected to the newly created bike path page, where the recorded path is displayed.

[UC4] Record path automatically



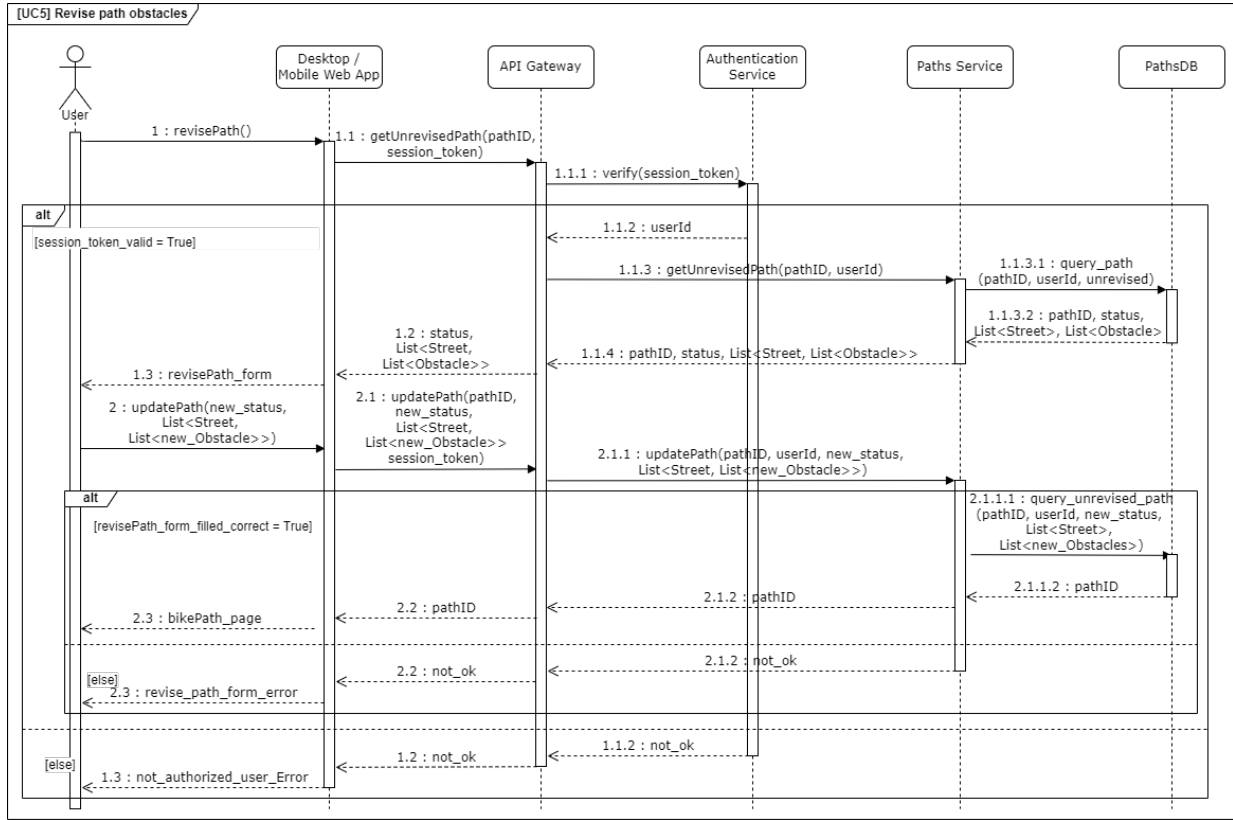
The sequence diagram for the Record Path Automatically use case begins when the user selects the "Automatic record" option, available in the navbar by choosing "Record" and then "Automatic" (Figure 15). After entering the "Automatic Record" page (Figure 18) and clicking Record, the **client application (mobile only)** checks the availability of GPS and sensor data required to perform the recording. If GPS or sensor data are unavailable, the use case is terminated.

Once the external components are verified, the **mobile application** sends an automatic record request to the **API**. The **API** first invokes the **Authentication Service** to verify that the user is authenticated, since this functionality is available only to registered users of the BBP platform. If the authentication is successful, the API delegates the request to the **Automatic Path Recording Service**, which starts the communication with the **Message Broker** to that User, where the raw data will be sent to the **Statistics, Path Construction and Obstacle Detection Service**. Since these services mainly perform computation and exchange data through the broker, they are collectively referred to as **Computation Services**.

At this point, the **API** confirms the User that the recording has started, and a Web-Socket communication channel between the **mobile application**, the **API**, and the **Automatic Path Recording Service** is established. Through this channel, GPS and sensor data are continuously sent to the **Message Broker**, allowing the **Computation Services** to process the stream and produce the results to be stored.

When the user stops recording at the end of the path, the **mobile application** sends a message through the WebSocket connection to terminate the recording session. The **Computation Services** then publish their final results to the **Message Broker**. This output is collected by the **Automatic Path Recording Service** and forwarded to the **API**. The **API** delegates the persistence task to the **Paths Service**, which stores the recorded path and returns a *PathID*, which allows the client to be redirected to the newly created bike path page, where the recorded path is displayed

[UC5] Revise path obstacles



To focus on the components' runtime behavior, the diagram assumes that the user has already recorded a path automatically (as shown in the sequence diagram for [UC4] Record path automatically). The *revisePath()* method redirects the user to the "Automatic Record" page, where the recorded data is displayed. Since a user can have only one automatically recorded path at a time, starting a new automatic recording requires first reviewing the existing one and saving it correctly.

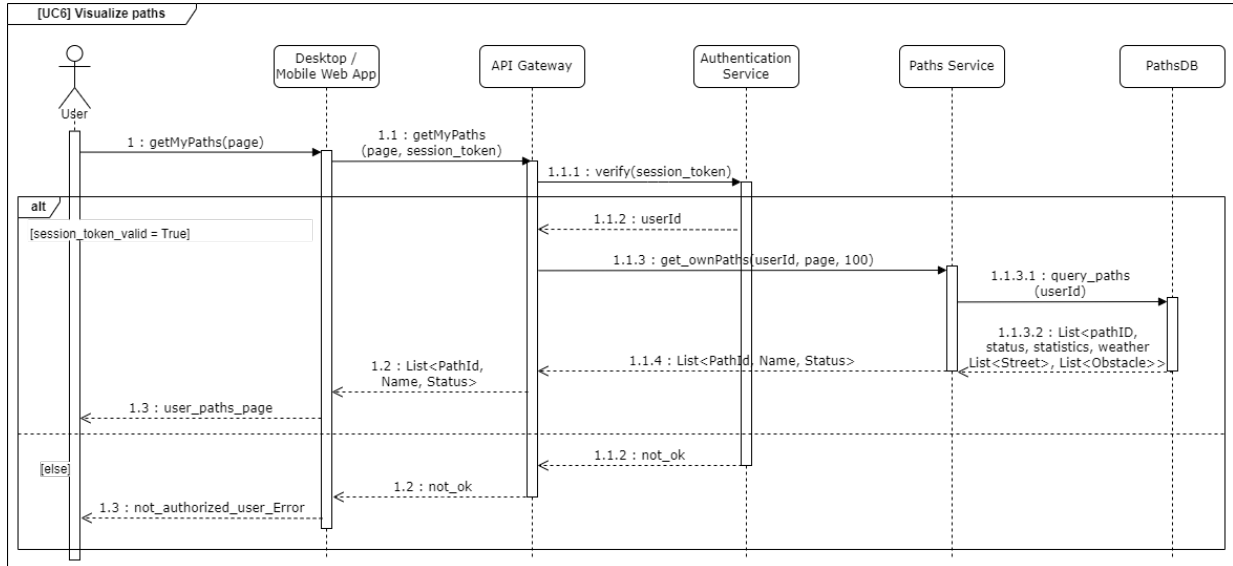
The sequence diagram for the Revise Path Obstacles use case then begins when the user requests the recorded data on the "Automatic Record" page after completing an automatic recording (Figure 18). Here, the **client application** (either desktop or mobile) sends a path query to the **API**.

The **API** first invokes the **Authentication Service** to verify that the user is authenticated, since this functionality is available only to registered users of the BBP platform. If authentication is successful, the **API** delegates the request to the **Paths Service**, which retrieves the unrevised path data and formats it appropriately. The formatted data are returned to the **API** and then to the client, so the user can review and edit it.

After reviewing the data, the user applies the necessary modifications based on their knowledge of the path and submits a revise path request through the client application. The **API** then invokes the **Paths Service**, which validates the updated information and,

if valid, updates the stored path (*query_unrevised_path()* performs different deletion and update over the path depending if it is needed to delete obstacles), marks it as revised, and returns a *PathID*, which allows the client to be redirected to the newly created bike path page, where the recorded path is displayed

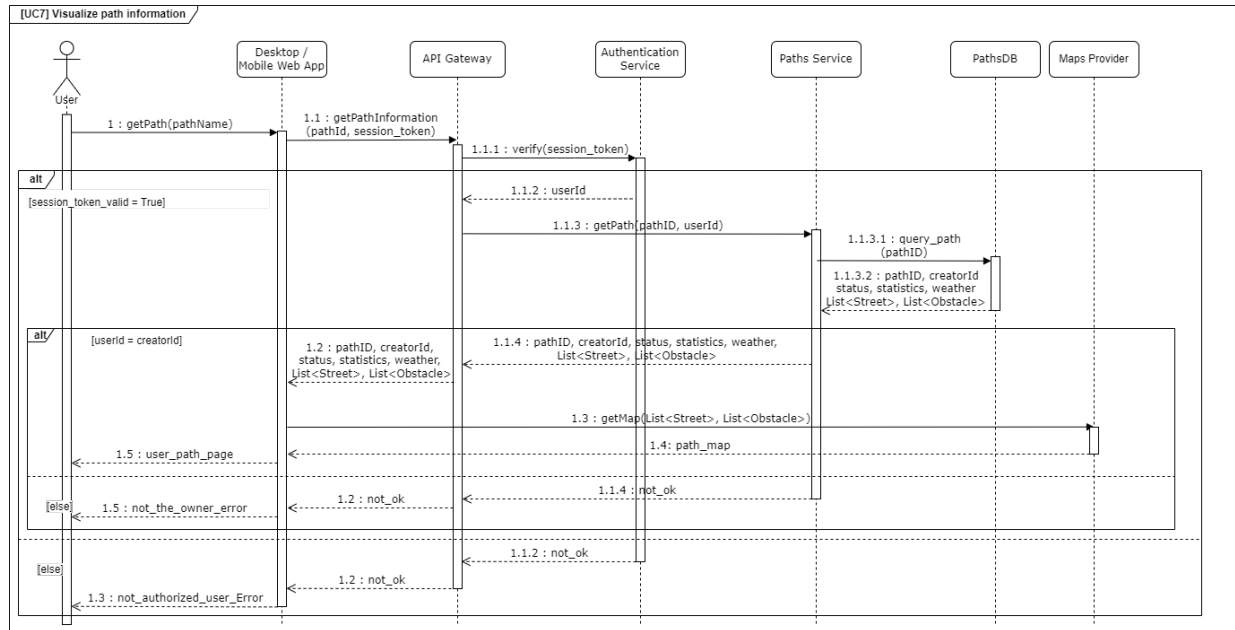
[UC6] Visualize paths



The sequence diagram for the Visualize paths use case begins when the user selects "My record" on the navbar (Figure 5). The **client application** (either desktop or mobile) then sends a query path request to the **API**.

The **API** first invokes the **Authentication Service** to verify that the user is authenticated and to retrieve the *UserID* associated with the User, since this functionality is available only to registered users of the BBP platform. If authentication is successful, the **API** delegates the request to the **Paths Service**, which uses the *UserID* to retrieve and return the list of paths recorded by the user on the "My Records" page (Figure 11).

[UC7] Visualize path information

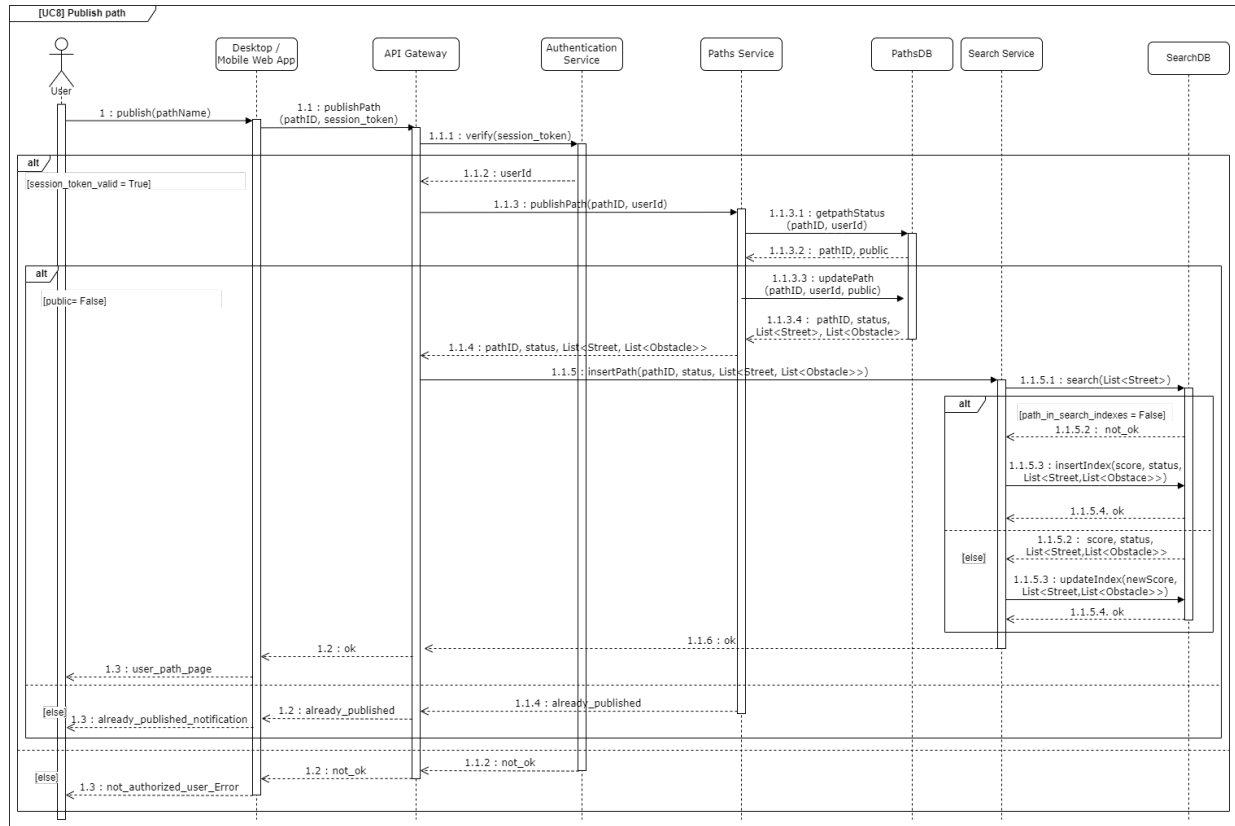


To focus on the components' runtime behavior for this specific use case, the diagram assumes that the user has already accessed the "My Records" page ((Figure 11)), where their recorded paths are listed (as shown in the sequence diagram for [UC6] Visualize paths). The user then selects a path by clicking its identifying name, which triggers *getPath(pathName)*. At this point, the runtime view of the use case begins.

The sequence diagram for the Visualize path information use case then begins when the user requests information about a recorded path. The **client application** (either desktop or mobile) sends a path query request to the **API**.

The **API** first invokes the **Authentication Service** to verify that the user is authenticated and to retrieve the *UserID* associated with the User, since this functionality is available only to registered users of the BBP platform. If authentication is successful, the **API** delegates the request to the **Paths Service**, which uses the *UserID* and the *PathID* to retrieve the selected path information and return it to the **API**, and then to the **client application**. At this point, the **Maps service** is invoked to retrieve and render the map associated with the path (Only if it is available, ignored in the diagram for simplicity). Finally, the path information and the map are displayed on the My Record page (Figure 13).

[UC8] Publish path



To focus on the components' runtime behavior for this specific use case, the diagram assumes that the user has already accessed the "My Records" page (Figure 11) and selected a recorded path by clicking its identifying name. This redirects the user to the "My Record" page (Figure 13), where the Publish option is available, and triggers *publish(pathName)* (Figure 14).

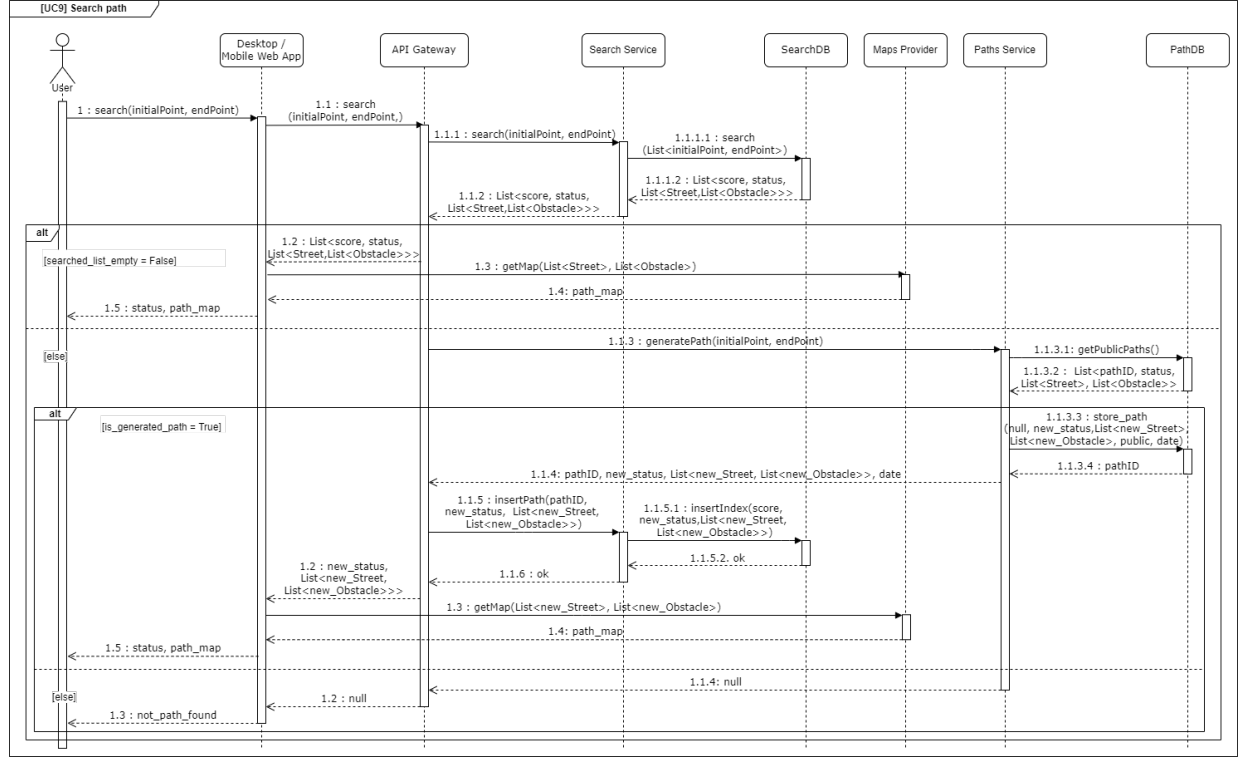
The sequence diagram for the Publish path use case then begins when the user clicks the publish button for an specified path with an associated *pathID*. The **client application** (either desktop or mobile) sends a publish path request to the **API**.

The **API** first invokes the **Authentication Service** to verify that the user is authenticated and to retrieve the *UserID* associated with the User, since this functionality is available only to registered users of the BBP platform. If authentication is successful, the **API** then delegates the request to the **Paths Service**, which uses the *UserID* and the *PathID* to retrieve the selected path and verify that it is not already public. If the publication is valid, the **Paths Service** returns reduced path information (Status and streets with obstacles) to the **API**.

At this point, the **API** invokes the **Search Service** to update the path status in the search engine. The **Search Service** first checks whether an equivalent path already exists in the **search database**. If the path is new, it computes an initial score and stores the

path together with its score. Otherwise, it computes an updated score based on the existing score and the new information, and updates the path entry in the database

[UC9] Search path



The sequence diagram for the Search path begins when the user selects an origin and a destination point and clicks "Search" in the homepage, the **client application (either desktop or mobile)** sends a search path request to the **API**, including the origin and destination as parameters.

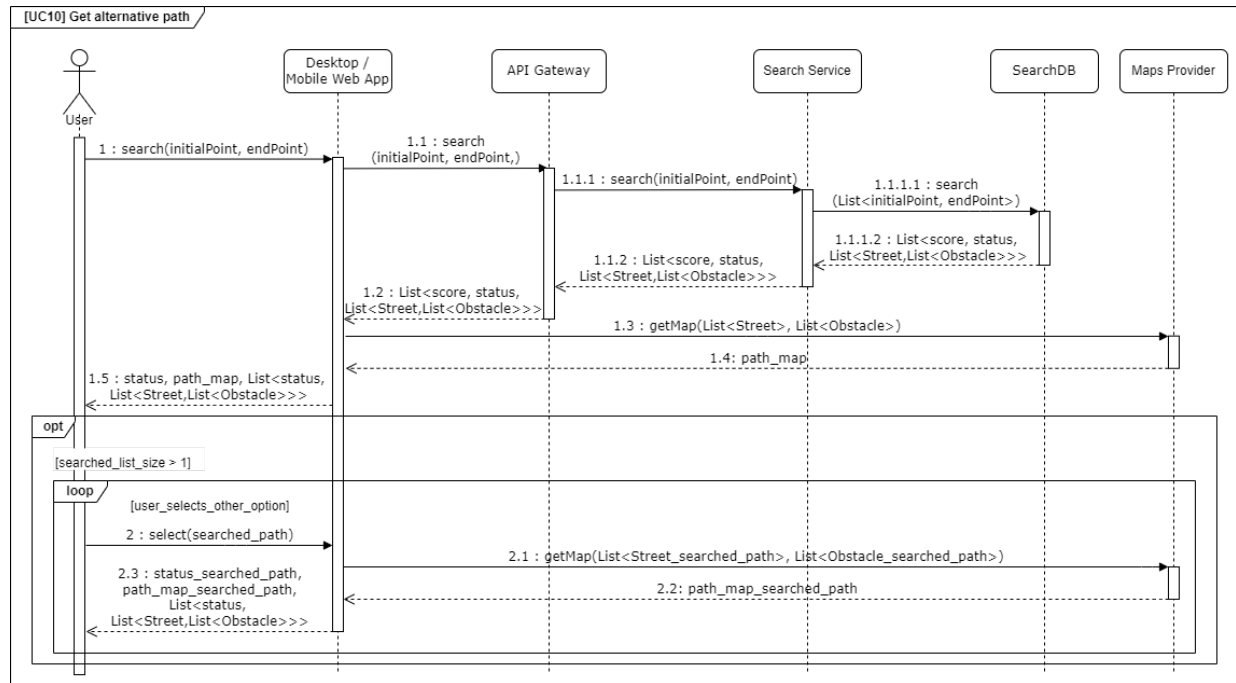
In this case, the **API** does not invoke the **Authentication Service**, since this functionality is available to any user of the BBP platform. The **API** then invokes the **Search Service**, which queries the **Search Database** to retrieve all indexed paths that match the search parameters, ordered by score. If the **Search Database** returns results, they are sent back to the **API** and then to the **client application**.

If the search returns no results, the **API** invokes the **Paths Service** to try to generate a new path with the public information in the BBP Platform. The **Paths Service** retrieves all the public paths and, based on their streets (segments), attempts to construct a route from the origin to the destination. If a path can be generated, it is returned to the **API**. At this point, the **Search Service** is invoked again to insert the new path into the **Search Database** with a newly computed overall score.

Finally, once the results are returned to the **client application**, the **Maps service** is invoked to retrieve and render the map associated with the path with the best score in the

result list (Only if it is available, ignored in the diagram for simplicity). The map and the list of alternative paths are then displayed on the “Search” page (Figure 9).

[UC10] Get alternative path



To focus on the components’ runtime behavior for this specific use case, the diagram presents a simplified version of the search functionality. The complete runtime view is shown in the previous diagram ([UC9] Search Path); here, the focus is on the list of results returned after the search is performed.

The sequence diagram for the Get alternative path then begins when the user selects an origin and a destination point and clicks “Search” in the homepage, the **client application (either desktop or mobile)** sends a search path request to the **API**, including the origin and destination as parameters.

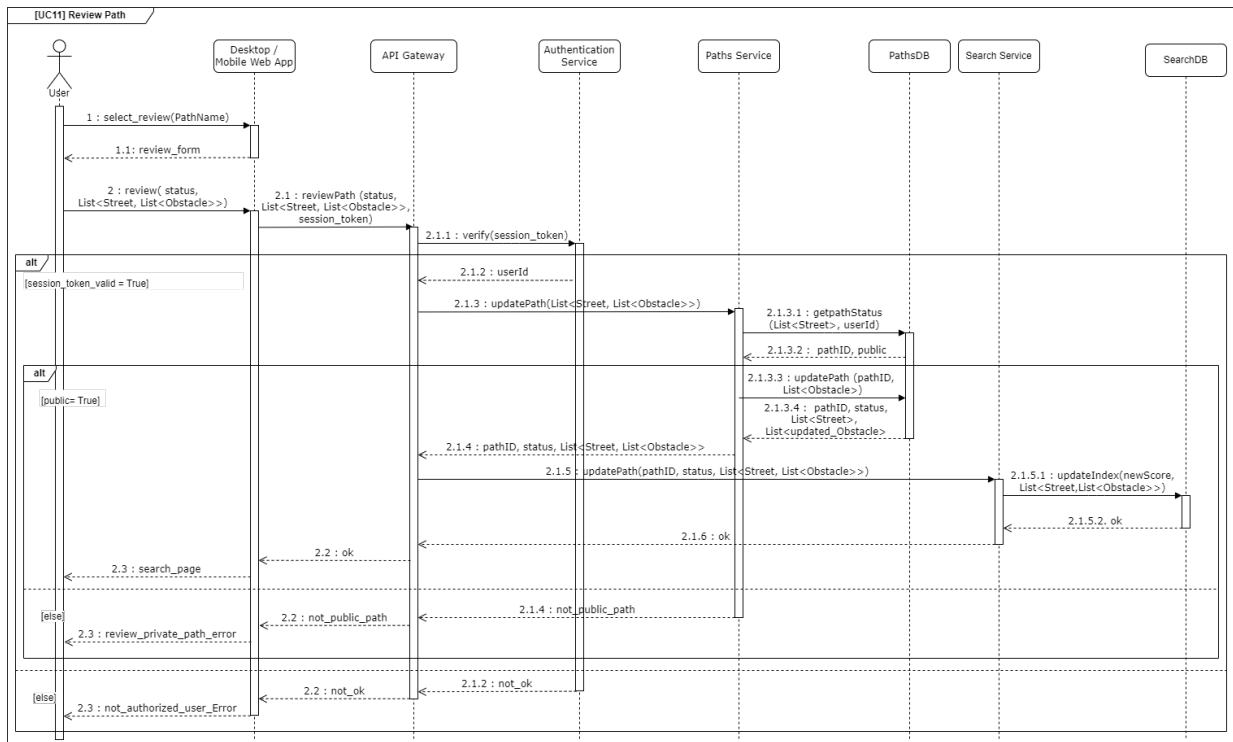
The **API** invokes the **Search Service**, which queries the **Search Database** to retrieve all indexed paths that match the search parameters, ordered by score. If the **Search Database** returns results, they are sent back to the **API** and then to the **client application**.

Once the results are returned to the **client application**, the **Maps service** is invoked to retrieve and render the map associated with the path with the best score in the result list (Only if it is available, ignored in the diagram for simplicity). The map and the list of alternative paths are then displayed on the “Search” page (Figure 9).

If the user selects a different path from the alternative list, the **client application (either desktop or mobile)** invokes the **Maps service** again to render a new map for the selected path. The updated view is then presented to the user, showing the new map

and an updated list of alternatives that includes the initial option and excludes the newly selected one.

[UC11] Review path



To focus on the components’ runtime behavior for this specific use case, the diagram assumes that the user has searched for a path by specifying an origin and a destination, obtaining a list of recommended paths (as shown in the sequence diagrams for [UC9] Search Path and [UC10] Get alternative path). The user then selects a path by clicking its identifying name and clicks the Review button, which triggers *select_review(pathName)*. At this point, the runtime view of the use case begins.

The sequence diagram for the Review path use case then begins when the user requests the Review form on the search results page (Figure 10). After the user completes and submits the form, the **client application** (either desktop or mobile) sends a path update request to the **API**.

The **API** first invokes the **Authentication Service** to verify that the user is authenticated, since this functionality is available only to registered users of the BBP platform. If authentication is successful, the **API** delegates the request to the **Paths Service**, which first checks whether the reviewed path is public. If it is, the service updates the path obstacles and records the update date. A reduced representation of the updated path

information is then returned to the **API**

Finally, the **API** invokes the **Search Service**, which updates the index associated with the reviewed path by computing a new score based on the existing score and the newly provided information (updated obstacles and review status).

2.5 Component interfaces

This section provides a comprehensive summary of all methods within the interfaces of the Best Bike Paths (BBP) system. The system employs a **hybrid communication architecture**:

- **Internal REST APIs:** For request-response operations between services (e.g., Path Service, Search Service, Authentication Service)
- **WebSocket Protocol:** For real-time bidirectional communication (e.g., sensor data streaming during path recording)
- **Event-Driven Messaging:** For asynchronous processing and inter-service communication

API

- **register(name, email, password):** Validates the request and forwards it to the Auth Service.
 - Returns session_token (success)
 - Error: not_ok with error messages pointing out the error (server error or malformed fields)
- **login(email, password):** Validates the request and forwards it to the Auth Service.
 - Returns session_token (success)
 - Error: not_ok with error messages pointing out the error (server error or malformed fields)
- **createPath(status, List<Street>, session_token):** Receives the create-path request and the path metadata from the Web App and orchestrates the token verification and the path creation.
 - Returns pathID (success)
 - Error: not_ok with error messages pointing out the error “Unauthorized: invalid or expired session token,” or “Invalid request: missing status/streets.”
- **automaticRecord(mobile_sensor_status, GPS_status, session_token):** Receives the start-record request and triggers verification and start.
 - Returns ok (success)

- Error: `not_ok` with error messages pointing out the error “Invalid request: missing/invalid `session_token` or sensors not ready.”
- **`send(actual_location, sensor_data, userId)`**: Streams sensor data to the automatic recording service during the recording loop.
 - Returns `ok` (success)
 - Error: `not_ok` with error messages pointing out the error “Stream rejected: missing fields or malformed payload.”
- **`sendStopSignal(userId)`**: Requests recording stop and finalization.
 - Returns `ok` (success)
 - Error: `not_ok` with error messages pointing out the error “Stop failed: recording not active or services unavailable.”
- **`getUnrevisedPath(pathID, session_token)`**: Forwards request to validate session and fetch the unrevised path details.
 - Returns `pathID`, `status`, `List<Street>`, `List<Obstacle>`
 - Error: `not_ok` with error messages pointing out the error “Unauthorized (invalid/expired token) or invalid `pathID`.”
- **`updatePath(pathID, new_status, List<Street>, List<Obstacle>, session_token)`**: Forwards request to validate session and update the path.
 - Returns `pathID` (updated path id confirmation)
 - Error: `not_ok` with error messages pointing out the error “Unauthorized (invalid token) or invalid update request.”
- **`getMyPaths(page, session_token)`**: Forwards the request to authenticate the session and fetch the user’s paths for the given page.
 - Returns `List<PathId, Name, Status>` (the page of results)
 - Error: `not_ok` with error messages pointing out the error “Unauthorized (invalid token).”
- **`requestInfoPath(pathId, session_token)`**: Forwards the “get path details” request, ensuring the session is verified and the path info is fetched.
 - Returns `pathID`, `creatorId`, `status`, `statistics` (when available), `weather` (when available), `List<Street>`, `List<Obstacle>`
 - Error: `not_ok` with error messages pointing out the error “Unauthorized (invalid token).”
- **`publishPath(pathID, session_token)`**: Forwards the publish request, verifies the session, and triggers publishing in services.

- Returns ok (publish completed)
- Error: not_ok with error messages pointing out the error “Unauthorized (invalid token).”
- **search(initialPoint, endPoint):** Forwards the search request to Search Service and returns the results to the client.
 - Returns List<score, status, List<Street>, List<Obstacle>>
 - Error: not_ok with error messages pointing out the error “Invalid coordinates or search service unavailable.”
- **reviewPath(status, List<Street>, List<Obstacle>, session_token):** Entry point for submitting a review; verifies the user session then updates the path.
 - Returns ok (success)
 - Error: not_ok or not_public_path with error messages “Invalid request (missing fields) or invalid/expired session token.”

Auth

- **register(name, email, password):** Validates the data, hashes the password, and creates the user in UsersDB.
 - Returns session_token (success)
 - Error: not_ok with error messages pointing out the error (server error or malformed fields)
- **login(email, password):** Looks up the user by email, compares stored hashed_password with hash(password), and creates a session if they match.
 - Returns session_token (success)
 - Error: not_ok with error messages pointing out the error (server error or malformed fields)
- **verify(session_token):** Verifies the session_token and, if valid, returns the associated userId to authorize the operation.
 - Returns userId (success)
 - Error: not_ok with message like “Invalid session token.”

Paths

- **createPath(userId, status, List<Street>), statistics (optional), weather (optional):** Creates a new bike path for a specific user by storing the path data and returning the created identifier.
 - Returns pathID (success)

- Error: not_ok with message like “Invalid session token.”
- **getUnrevisedPath(pathID, userId):** Retrieves the path data that can be revised (streets/obstacles/status) for that user and path.
 - Returns pathID, status, List<Street>, List<Obstacle>
 - Error: not_ok with error messages pointing out the error “Path not found or cannot be revised.”
- **updatePath(pathID, status, List<Street>, List<Obstacle>):** Applies the revision (updates path fields and obstacle list) and persists data.
 - Returns pathID
 - Error: not_ok with error messages pointing out the error “Update failed: invalid data or storage failure.”
- **getOwnPaths(userId, page, limit):** Retrieves the user’s paths with a number limit.
 - Returns List<PathId, Name, Status>
 - Error: not_ok with error messages pointing out the error “Update failed: invalid data or storage failure.”
- **getPath(pathID, userId):** Retrieves the path details for the requested path.
 - Returns pathID, creatorId, status, statistics, weather (when applicable), List<Street>, List<Obstacle>
 - Error: not_ok with error messages pointing out the error “Path not found.”
- **publishPath(pathID, userId):** Marks the path as public (if not already public) and returns the path details needed for insertion into the public index.
 - Returns pathID, status, List<Street>, List<Obstacle>
 - Error: already_public (“This path is already published”) or not_ok (“Path not found”)
- **generatePath(initialPoint, endPoint):** Generates a new path when no searched path is found.
 - Returns pathID, status, List<Street>, List<Obstacle>, date
 - Error: null and a message error “Could not generate a path for the given points.”

Search

- **insertPath(pathID, status, List<Street>, List<Obstacle>):** Inserts the published path into a searchable/indexed store (so others can find it).
 - Returns ok (success)

- Error: `search_error` with error messages pointing out the error “Indexing failed: could not insert published path.”
- **search(initialPoint, endPoint):** Searches for candidate paths between two points using the search index.
 - Returns `List<score, status, List<Street>, List<Obstacle>>`
 - Error: `search_error` with error messages pointing out the error “Search failed (index unavailable).”

SensorData

- **send(actual_location, sensor_data, userId):** Streams sensor data to the automatic recording service during the recording loop. This is done via the WebSocket Protocol.
 - Returns `ok` (success)
 - Error: `not_ok` with error messages pointing out the error “Stream rejected: missing fields or malformed payload.”

AutoPath

- **startRecording(userId):** Starts a recording session for the user. This is done via the WebSocket Protocol.
 - Returns `ok` (success)
 - Error: `recording_error` with error messages pointing out the error “Cannot start recording or initialization failed.”
- **send(actual_location, sensor_data, userId):** Takes the recording data and forwards it to the message broker. This is done via the WebSocket Protocol.
 - Returns `ok` (success)
 - Error: `recording_error` with error messages pointing out the error “Invalid data / ingestion failure.”
- **send(stop_signal, userId):** Stops recording, triggers the final outputs and returns the path with all its metadata.
 - Returns `Returns status, List<Street, List<Obstacle>>, Statistics, Weather`
 - Error: `recording_error` with error messages pointing out the error “Stop failed: no active recording for user.”

AutoRecord

- **send(events, actual_location, sensor_data):** Publishes events and data for consumers. This is done via the Message Broker.

- Returns ok (success)
- Error: `broker_error` with error messages pointing out the error “Publish failed / queue full / broker unavailable.”
- **pool():** Retrieves computed data from the specialized services. This is done via the Message Broker.
 - Returns ok (success)
 - Error: `broker_error` with error messages pointing out the error “Broker unavailable.”

DataProcess

- **pool():** Retrieves data/events from the broker for processing. This is done via the Message Broker.
 - Returns ok (success)
 - Error: `compute_error` “No events available / broker unreachable.”
- **send(results, statistics, bikepath, obstacles):** Publishes computed outputs (streets/obstacles/statistics) back into the flow. This is done via the Message Broker.
 - Returns ok (success)
 - Error: `compute_error` “Computation failed / publish failed”

Weather

- **getStatus():** Checks if the weather integration is active/available.
 - Returns `weather_service_status` (success)
 - Error: `weather_error` with error messages stating the error “Weather service unavailable.”
- **getWeather():** Retrieves weather conditions to attach to the created path (optional branch).
 - Returns `weather_conditions` (success)
 - Error: `weather_error` with error messages stating the error “Could not fetch weather (timeout/invalid location).”

GPS

- **getStatus():** Checks whether GPS is available/active.
 - Returns `GPS_status` (success)
 - Error: `GPS_error` with error messages stating the error “GPS unavailable / permission denied / no fix.”

- **getLocation():** Retrieves the current position used during recording.
 - Returns Actual_Location (success)
 - Error: GPS_error with error messages stating the error “GPS unavailable / permission denied / no fix.”

Sensor

- **getStatus():** Checks whether required sensors are available/active.
 - Returns mobile_sensor_status (success)
 - Error: sensor_error with error messages stating the error “Sensors unavailable / permission denied.”
- **getSensorData():** Retrieves sensor readings used during recording.
 - Returns Sensor_Data (success)
 - Error: sensor_error with error messages stating the error “Sensor read failed.”

Maps

- **getMap(List<Street>, List<Obstacle>):** Generates a map representation for the path based on streets and obstacles.
 - Returns path_map (success)
 - Error: map_error with error messages stating the error “Provider error: could not generate map.”

PathDB

- **store_path(userId, status, List<Street>, date, statistics (optional), weather (optional))** Persists the recorded path and its metadata (userId, status, list of streets, date), including statistics and weather when available and returns the generated id.
 - Returns pathID (success)
 - Error: db_error with error messages stating the error like “Database error: could not create user (unique constraint violation or connection issue).”
- **query_path(pathID, userId (optional), unrevised (optional))**: Reads a path record (can also be the unrevised version for updating) from storage.
 - Returns pathID, creatorId, status, statistics, weather, List<Street>, List<Obstacle>
 - Error: db_error with error messages stating the error like “Database error: could not retrieve path (connection/query failure).”
- **query_unrevised_path(pathID, userId, status, List<Street>, List<Obstacle>)**: Persists the revised data (despite the name, it’s basically an update operation in the diagram).

- Returns pathID
- Error: db_error with error messages stating the error like “Database error: could not update path (write failed).”
- **query_paths(userId):** Queries stored paths that belong to the given user.
 - Returns pathID, status, List<Street>, List<Obstacle>
 - Error: db_error with error messages stating the error like “Database error: could not retrieve paths.”
- **updatePath(pathID, userId, public (boolean)):** Updates the path’s privacy status to public.
 - Returns pathID, status, List<Street>, List<Obstacle>
 - Error: db_error with error messages stating the error like “Database error: could not update path status.”
- **getPathStatus(pathID, userId):** Retrieves path’s privacy status.
 - Returns pathID, public or not (boolean)
 - Error: db_error with error messages stating the error like “Database error: could not retrieve path status.”
- **getPublicPaths():** Retrieves published paths.
 - Returns List[pathID, status, List<Street>, List<Obstacle>>
 - Error: db_error with error messages stating the error like “Database error: could not retrieve paths.”

UsersDB

- **insert(name, email, hashed_password):** Persists the new user record.
 - Returns Ok message
 - Error: db_error with error messages stating the error like “Database error: could not create user (unique constraint violation or connection issue).”
- **user(email):** Retrieves a user record by email for authentication checks.
 - Returns name, email and hashed_password
 - Error: db_error with error messages stating the error like “Database error: could not read user (connection/query failure).”

SearchDB

- **search(List<Street>, List<initialPoint, endPoint>)**: Checks whether an index entry exists for the path’s street set or coordinates.
 - Returns List<score, status, List<Street>, List<Obstacle>>
 - Error: not_ok with error messages pointing out the error “Search/index lookup failed.”
- **insertIndex(score, status, List<Street, List<Obstacle>>)**: Stores the index info used by search.
 - Returns ok (success)
 - Error: db_error with error messages pointing out the error “Database error: could not insert index data.”
- **updateIndex(score, List<Street, List<Obstacle>>)**: Updates the index info used by search.
 - Returns ok (success)
 - Error: db_error with error messages pointing out the error “Database error: could not update index data.”

2.6 Selected architectural styles and patterns

The BBP Platform is designed following a microservices architecture. In particular, the Automatic Path Recording Service adopts an event-driven approach by publishing and consuming events through a message broker. In general, the platform is organized using a three-layer logical separation.

- Three-layer architecture

The BBP Platform is organized into three functional layers. The *presentation layer* includes all user interfaces and communication components through which end users interact with the system. The *application layer* contains the platform’s business logic and computational services, implementing the core functionality of the system. Finally, the *data layer* include all the components responsible for storing, managing, and providing access to persistent information .

This separation is intended to decouple user interaction and communication concerns from the internal services offered by the platform, improving both flexibility and security. From a maintainability perspective, the separation of concerns allows changes to be introduced in any layer with minimal impact on the others, which is particularly valuable in systems that must evolve frequently. Additionally, the layered organization supports the application resilience and availability: failures in one layer are less likely to propagate and compromise the entire system, which simplifies recovery and reduces the overall impact of incidents. Finally, this clear logical layering enables

independent scalability strategies for each layer, allowing the platform to prioritize resources for the components that are most critical and frequently used, deriving in an efficient and smart division and usage of the resources.

- Microservices

The BBP platform is composed of many loosely coupled and independently deployable smaller components or services. Each service is specialized, has a well-defined responsibility, and communicates with other services through a combination of synchronous interfaces (APIs) and asynchronous communication via a message broker.

This architectural decision addresses similar objectives to those achieved by the layered style. In particular, the independence and single-responsibility nature of each service promotes a clear separation of business capabilities and enables service-specific strategies for resilience, availability, and fault isolation. From a deployment perspective, microservices support incremental and step-by-step releases, simplifying the establishment and evolution of the production environment. New functionality can be introduced, or existing behavior can be modified, by deploying or updating only the relevant services rather than redeploying the entire application. Finally, the microservice approach facilitates technological flexibility: each service can adopt the most appropriate programming language, framework, or infrastructure for its requirements, without constraining the entire platform to a single technology stack.

- Routing pattern

Adopting a microservices style introduces the challenge of managing service instance locations and addressing changes caused by replication and scaling. As described in the Deployment View, containerizing the services together with service orchestration enables the adoption of a service discovery mechanism based on DNS. The orchestration platform provides a DNS-based discovery, through which services can be registered under stable service names and their network addresses can be resolved dynamically at runtime. This approach supports elastic scaling by allowing instances to be added or removed transparently, while avoiding the need to introduce additional service-registry components and reducing overall operational complexity in the microservices environment.

- Resiliency pattern

Another aspect to consider when a microservice style is included is monitoring service health and detecting failures quickly, in order to identify unavailable services and react accordingly. Container orchestration platforms address this need by continuously tracking the status of service instances through health checks and monitoring signals. When an instance becomes unhealthy or fails, the orchestrator can automatically restart it and, if required, scale replicas to maintain the desired level of availability.

- Security pattern

To ensure secure access to platform services, the BBP platform design includes an API Gateway that acts as a central entry point between clients and the business services. The gateway enforces authentication and authorization policies supported by the Authentication Service, ensuring that each user can access only the resources and operations they are permitted to use. In this way, the gateway works as a gatekeeper, reducing the attack surface by preventing unauthorized requests and limiting unnecessary exposure of internal services.

In addition, the API Gateway centralizes external communication and define overall business logic. Client requests are authenticated and then forwarded to the appropriate services, which may involve calling multiple services in sequence to complete a given operation and aggregating the results. This design avoids direct client-to-service communication, improving control over data exposure, and enables consistent security and traffic management policies at a single point.

- Communication pattern

The BBP Platform must guarantee continuous communication to support the automatic path recording functionality. For this reason, asynchronous communication is essential for the business logic associated with this feature, as it enables reliable data exchange between computation services. This is achieved through a message broker adopting a publish–subscribe model, which allows multiple services to publish events and subscribe to the information they require without slowing down the application response.

In addition to asynchronous messaging, the platform also uses synchronous communication for operations that require immediate responses (Almost all the features). This is implemented through RESTful interfaces (as described in the Component Interfaces section), providing a simple and standardized interaction model that is widely supported and easy to integrate across services.

- Event-driven architecture

As discussed in the previous section, the automatic path recording functionality requires a continuous flow of information from the user side to the server side. To satisfy this requirement, the BBP Platform adopts an event-driven architecture style in this part of the system, using a message broker to enable asynchronous communication and web sockets to provide continuous flow of information. In particular, four microservices participate in the automatic recording task as event producers and consumers, publishing to and subscribing from two main topics, called Record (Get raw data and analyses it) and Result (Return the final information computed from the flow of data). This event-driven approach enables reactive computation, where the microservices responsible for computation can process incoming data as it arrives

and produce new events for subsequent tasks, such as path reconstruction, obstacle detection, and statistics generation.

2.7 Other design decisions

Besides the design elements related to architectural styles and patterns exposed in the previous section, there are some other design decisions taken, which focus on different components or techniques that provide support to different qualities attributes essential in the BBP platform design.

- Reverse Proxy

A reverse proxy sits between a client and a web server by handling the request and response on behalf of the server. In the BBP platform, the load balancer acts as a reverse proxy at the edge of the system, since it receives incoming user requests and forwards them to the appropriate API Gateway instance based on a load balancing algorithm. This design makes the platform more scalable and available by distributing traffic and preventing a single backend instance from becoming a bottleneck.

Beyond scalability, the reverse proxy inclusion provide an additional layer of security to the platform. By applying rate limits, and filtering suspicious requests, it reduces the exposure of internal services and helps mitigate common attacks, such as DDoS attacks. Likewise, the Reverse proxy can enable caching, reducing load on backend services and improving the platform's response time.

- Master-Slave Replication fro Databases

Considering the platform's usage context (Cyclist-oriented application) and its expected usage, master-slave replication is the replication technique chose for the data storage layer. In Master-Slave Replication, one server acts as the master and manages the main database, while other servers, known as slaves, replicate the data from the master. The slaves cannot make changes to the data and they only copy what the master has.

This approach improves performance and scalability for heave read workloads by distributing search queries across multiple replicas (Slaves), while keeping write operations centralized (Master), guaranteeing a strong consistency of the data. The main trade-off is that the master becomes a single point of failure. However, considering that the platform is expected to be used mostly for the search feature, and that write traffic (path recording) is expected to be moderate, master-slave replication provides a good balance between strongly controlled writes, high-throughput reads and simplicity to set up and maintain.

- Auto scaling for API and microservices

To maintain consistent availability and meet performance targets, the BBP platform adopts auto scaling. Auto scaling adjusts system capacity, adding new servers or compute instances (or terminates them) in accordance with resource demand and the auto scaling policies an organization has established.

Given the BBP platform's usage context, this approach fits well, since traffic is not expected to be constantly high, but it may change abruptly (for instance, cycling events or cycle competitions). Auto scaling provides high availability and stable performance during traffic peaks, while avoiding unnecessary compute costs during low-demand periods.

In addition, the use of container orchestration and service discovery provides compatibility with auto scaling. New service instances can be started automatically, registered through discovery mechanisms, and integrated into execution flow without manual intervention, reducing implementation and maintenance effort.

3 USER INTERFACE DESIGN

Here is shown a mockup of the web application UI. The images depict the main screens of the system, which include a login and registration interface, a home page where users can search bike paths by origin and destination even without an account, once logged it has dedicated sections for recording trips, either manually or automatically, with options available to edit both manual and automatic records like adding streets and/or obstacles. The application also provides a page for visualizing all recorded paths and another for reviewing individual trips shared by other people.



Figure 4: HomePage Not LoggedIn.

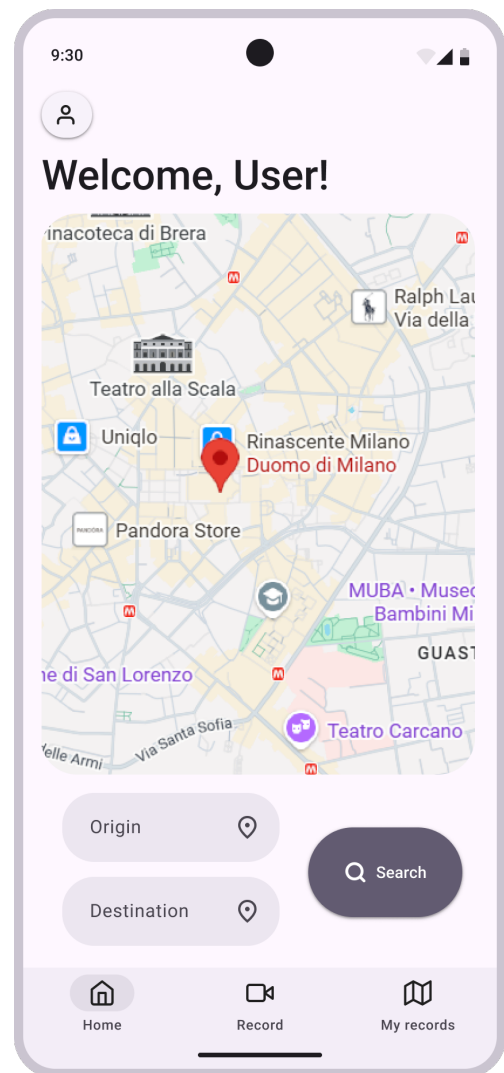
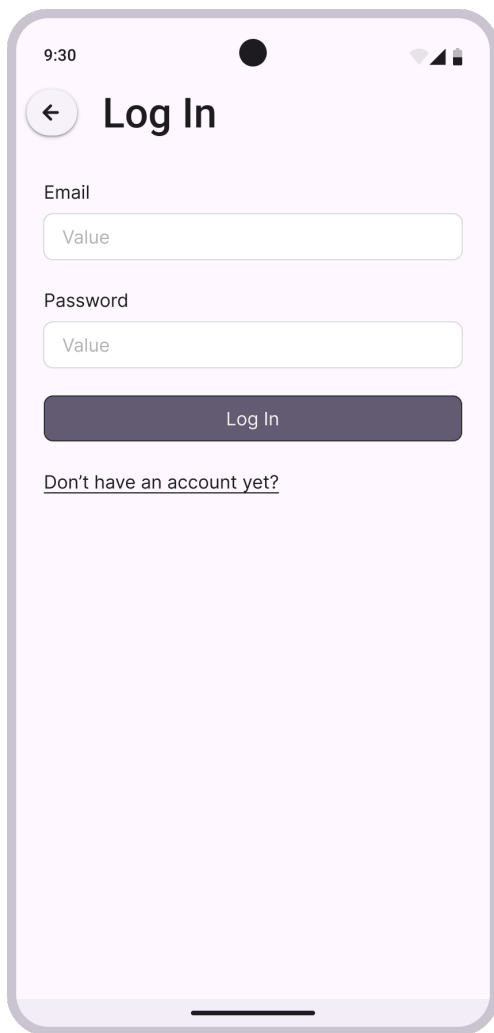


Figure 5: HomePage LoggedIn.



A mobile app login screen with a light purple background. At the top, the status bar shows 9:30, a black circle, and signal icons. Below the status bar is a header with a back arrow icon and the text "Log In". The main content area contains two input fields: "Email" and "Password", each with a "Value" placeholder. Below these fields is a dark purple button labeled "Log In". At the bottom, there is a link that says "Don't have an account yet?". The bottom of the screen features a home indicator bar.

9:30

← Log In

Email

Value

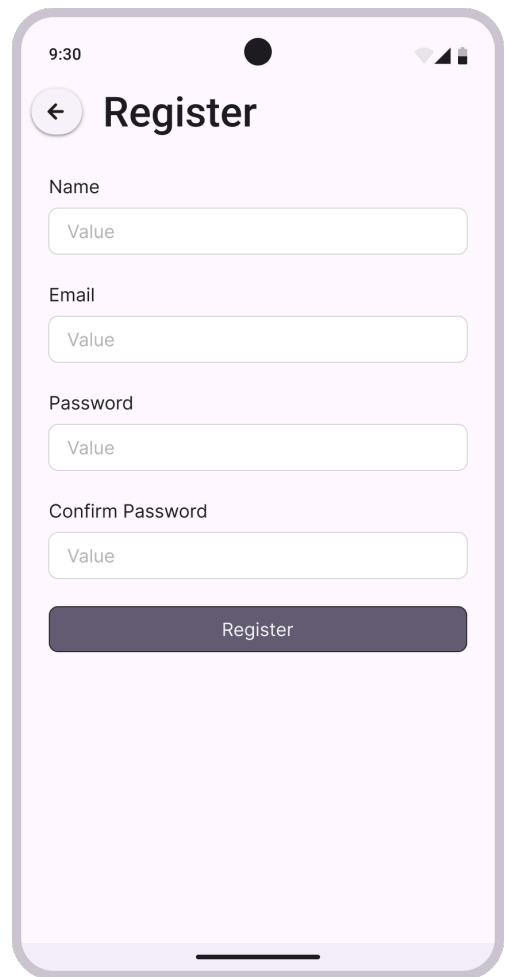
Password

Value

Log In

[Don't have an account yet?](#)

Figure 6: Login Page.



A mobile app register screen with a light purple background. At the top, the status bar shows 9:30, a black circle, and signal icons. Below the status bar is a header with a back arrow icon and the text "Register". The main content area contains four input fields: "Name", "Email", "Password", and "Confirm Password", each with a "Value" placeholder. Below these fields is a dark purple button labeled "Register". The bottom of the screen features a home indicator bar.

9:30

← Register

Name

Value

Email

Value

Password

Value

Confirm Password

Value

Register

Figure 7: Register Page.

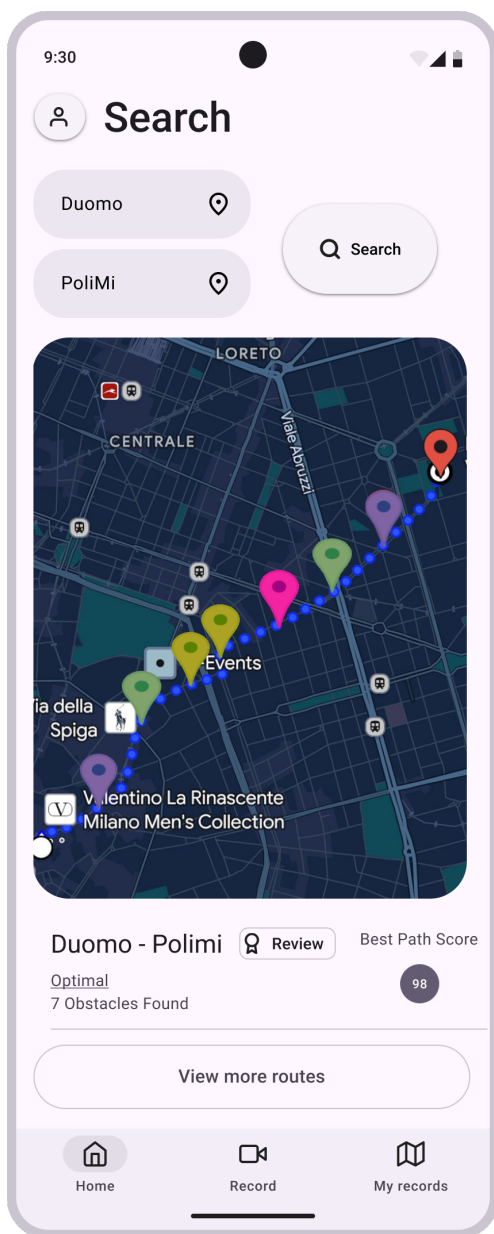


Figure 8: Search page.

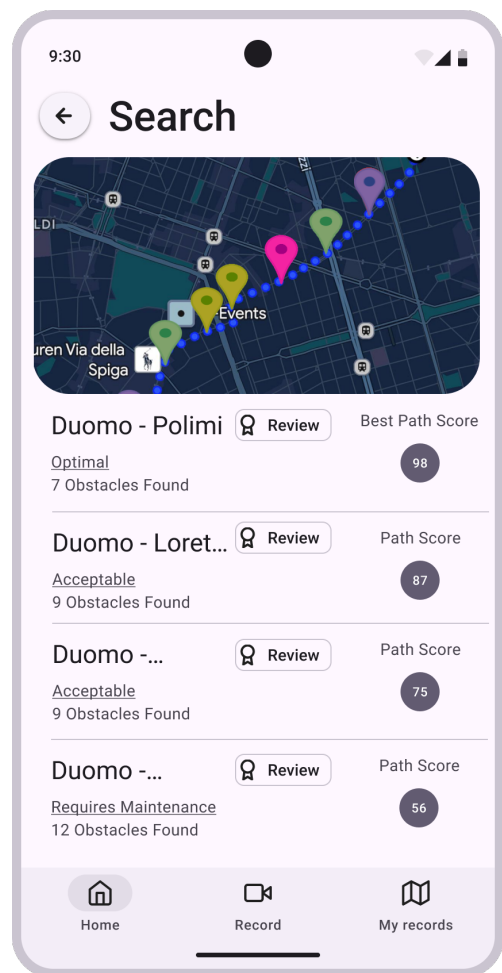


Figure 9: More routes page.

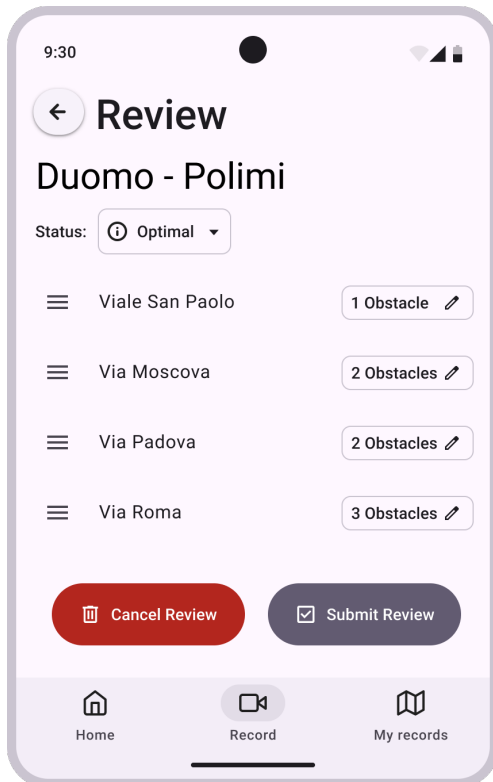


Figure 10: Review page.

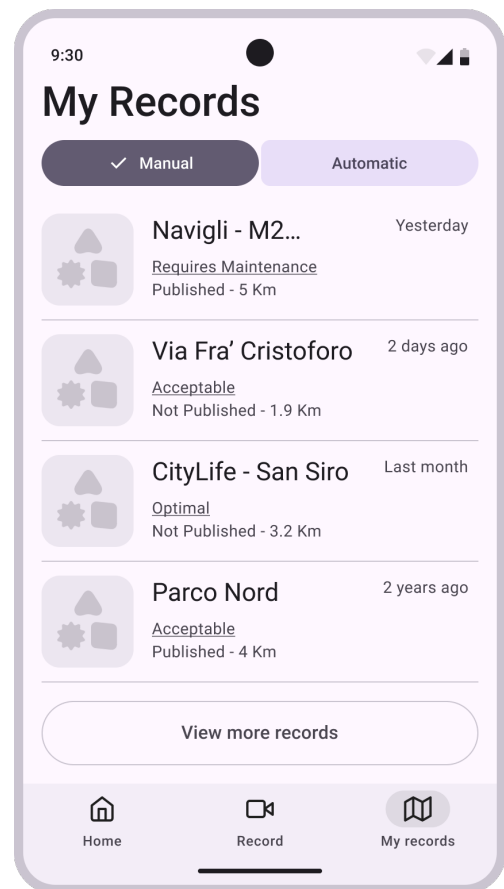


Figure 11: My Manual Records page.

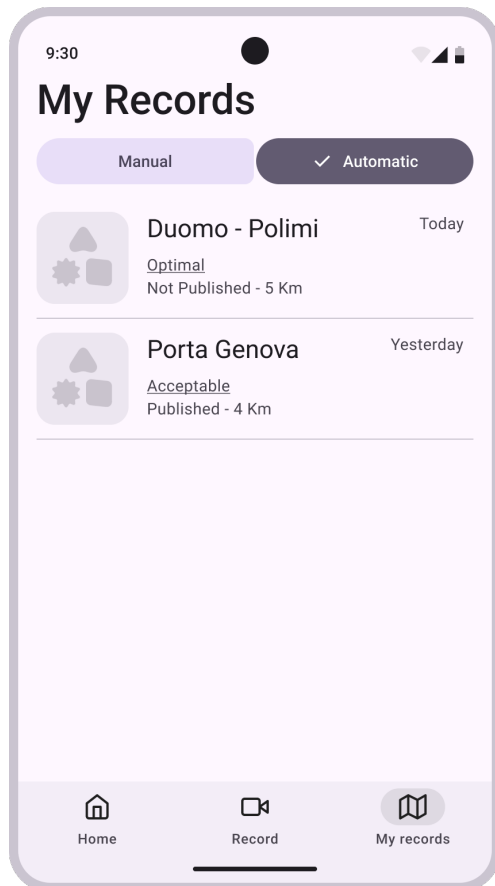


Figure 12: My Automatic Records page.

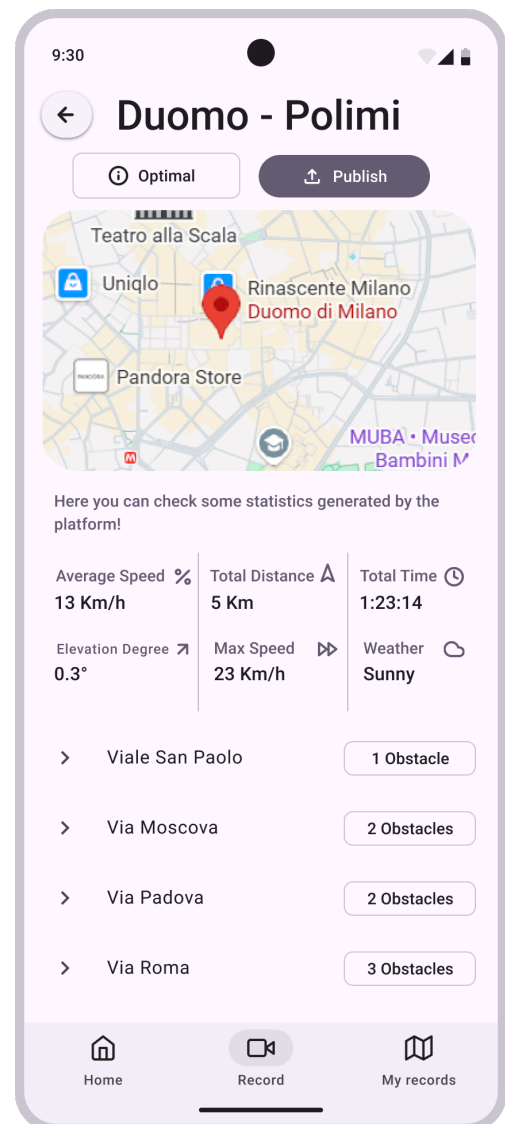


Figure 13: My record page.

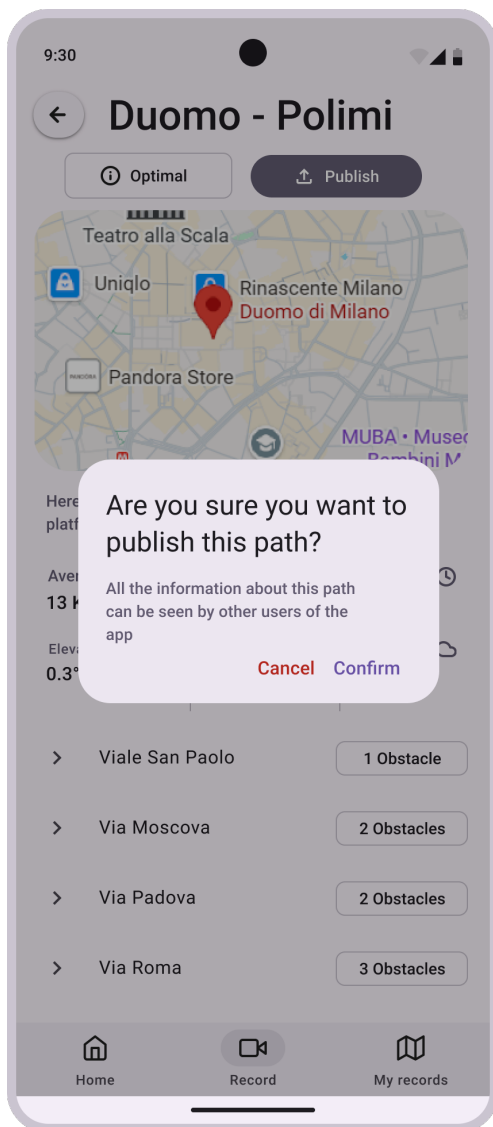


Figure 14: Publish window.

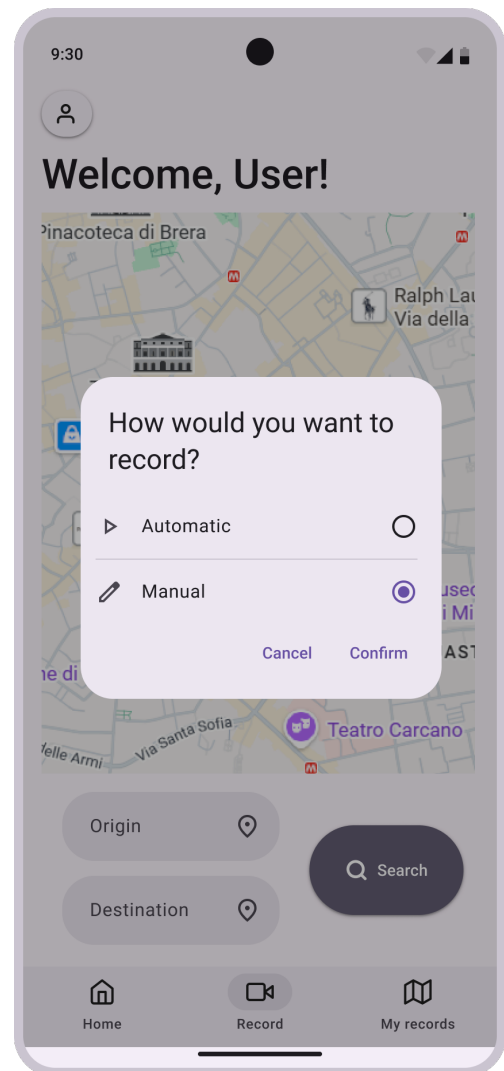


Figure 15: Record window.

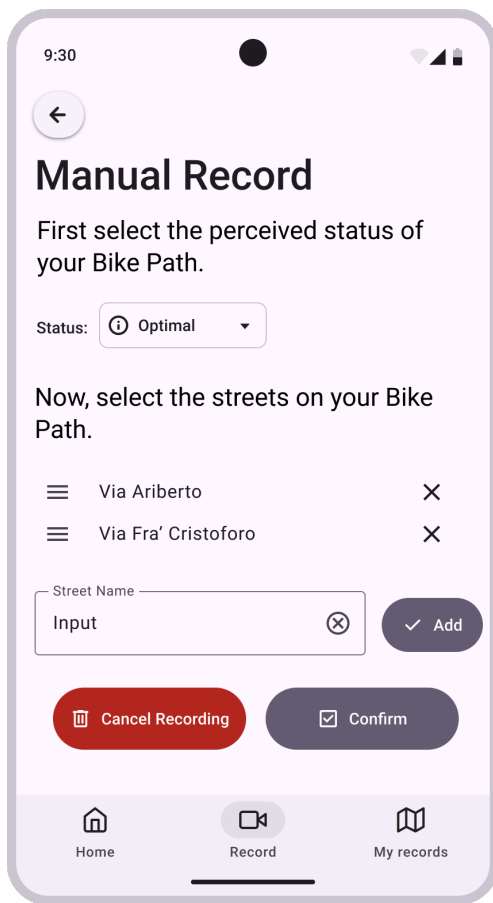


Figure 16: Manual record page.

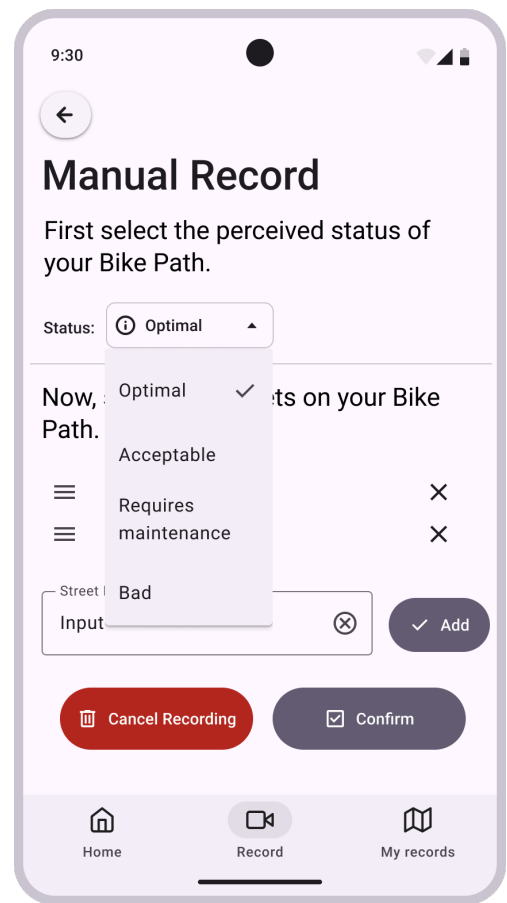


Figure 17: Status dropdown menu.

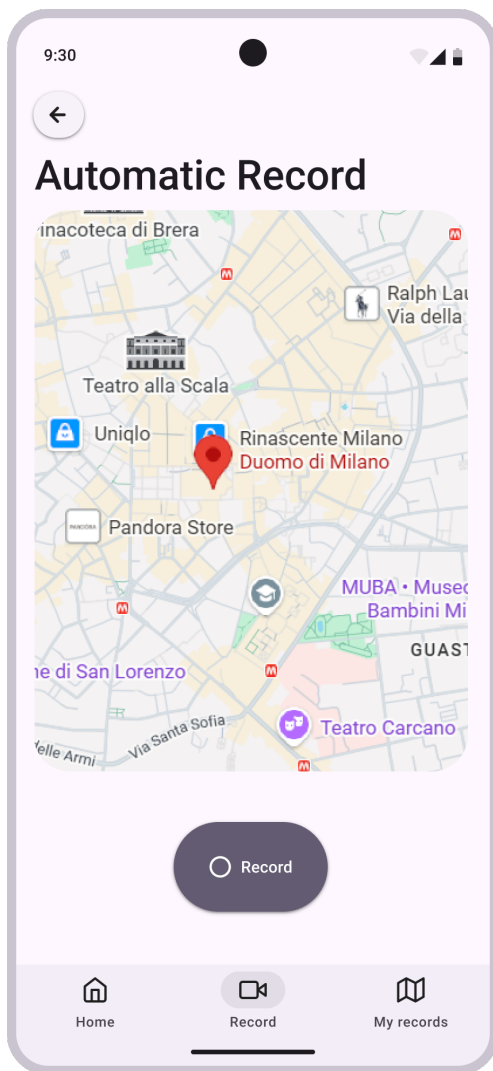


Figure 18: Automatic Record page.

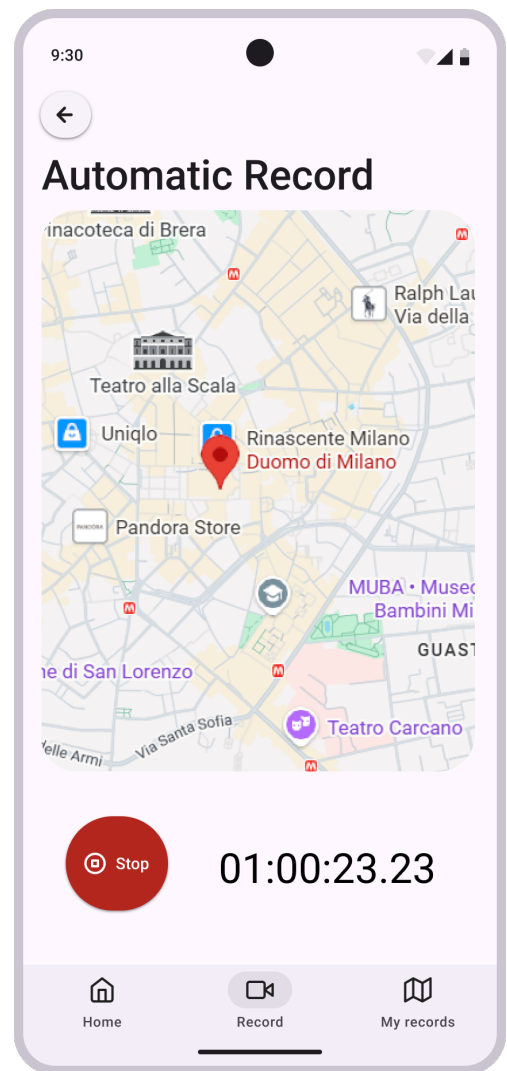


Figure 19: Recording page.

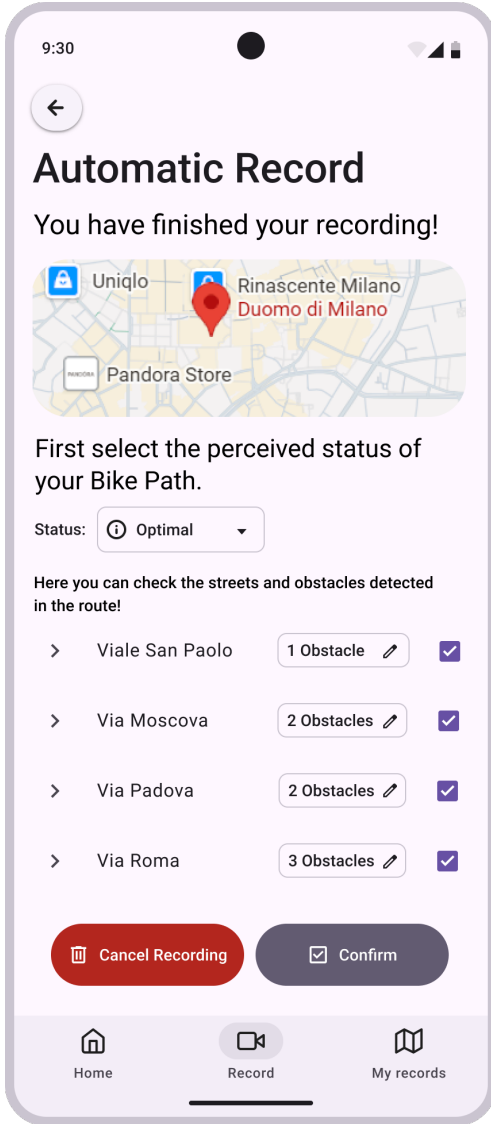


Figure 20: Recording Finish page.

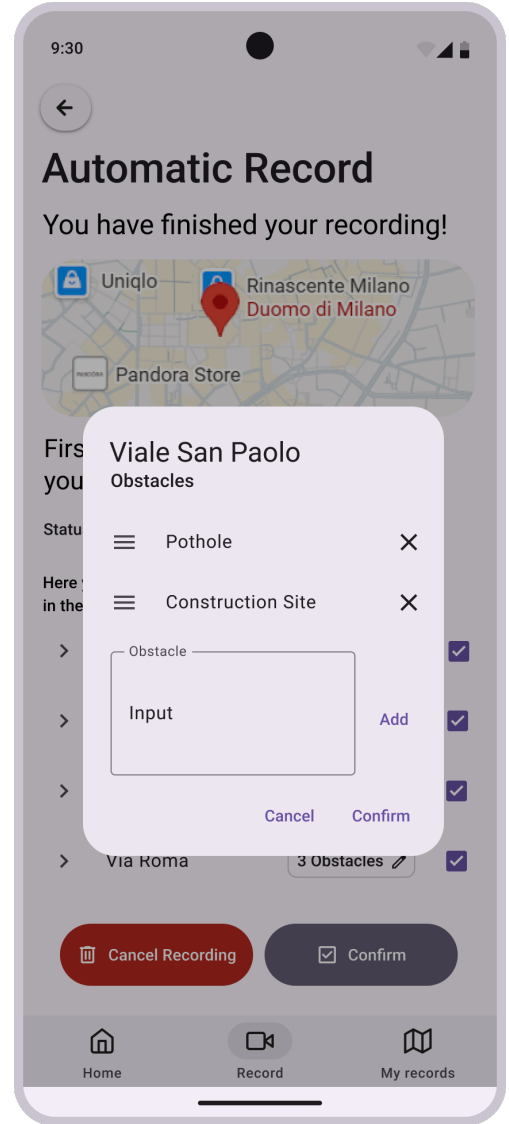


Figure 21: Obstacle Window menu.

4 REQUIREMENTS TRACEABILITY

This section provides traceability between the requirements defined in the RASD and the proposed design. For each requirement, we identify the set of design elements that contribute to satisfying it. The objective is to show that the architectural and detailed design choices are not arbitrary but are systematically derived from the system needs and shaped to meet the requirements as effectively as possible, while respecting the identified constraints and limitations.

To achieve this mapping, we present a table in which each row corresponds to a requirement and each column corresponds to a component. A cell marked with “X” indicates that the corresponding component contributes to satisfying that requirement (either by implementing it directly or by providing necessary supporting functionality).

Log in and register

- R1: When an unregistered user enters the BBP platform, it shall allow the user to create an account.
- R2: When a registered user enters the BBP platform, it shall allow them to log into their account.

	Desktop / Mobile Web App	Load Balancer	API Gateway	Authentication Service	UsersDB
R1	X	X	X	X	X
R2	X	X	X	X	X

Recording of paths

- R3: The BBP platform shall allow a logged-in user to manually record a bike path by providing the names of the streets on the bike path and their status.
- R4: When a logged-in user enters the BBP platform, it shall allow them to automatically record a bike path they are taking.
- R5: When a logged-in user automatically records a bike path, the BBP platform shall recognize obstacles on the path.
- R6: When a logged-in user with automatically recorded bike paths enters the BBP platform, it shall allow them to confirm the presence of obstacles automatically recognized in the recorded process.
- R7: When a logged-in user with automatically recorded bike paths enters the BBP platform, it shall allow them to correct automatically recognized obstacles in the path.

	Desktop Web App	Mobile Web App	Load Balancer	API Gateway
R3	X	X	X	X
R4	X	X	X	X
R5	X	X	X	X
R6	X	X	X	X
R7	X	X	X	X

	Authentication Service	UsersDB	Paths Service	PathsDB	Automatic Path Recording Service
R3	X	X	X	X	
R4	X	X	X	X	X
R5	X	X			X
R6	X	X	X	X	X
R7	X	X	X	X	X

	Message Broker	Statistics Service	Path Construction Service	Obstacle Detection Service
R3				
R4	X	X	X	X
R5	X		X	X
R6	X		X	X
R7	X		X	X

	Weather Service Provider	Web GPS Reciever	Mobile Sensor Manager
R3			
R4	X	X	X
R5			X
R6		X	X
R7		X	X

Managing of paths

- R8: The BBP platform shall show to a logged-in user their list of recorded paths.
- R9: The BBP platform shall provide a logged-in user the path statistics about their automatically recorded paths.
- R10: When the Weather service is available, the BBP platform shall provide weather information to a logged-in user for their automatically recorded paths.
- R11: When a logged-in user with recorded paths enters the BBP platform, it shall allow them to make their bike paths visible to other users who are searching for routes.
- R12: The BBP platform shall allow a logged-in user to insert new status and obstacles information to a public bike path.
- R13: When a logged-in user selects a previously recorded bike path, the BBP platform shall display a map view showing the selected bike path and its associated obstacles.
- R14: When a logged-in user selects a previously automatically recorded bike path, the BBP platform shall display the path statistics computed in the recording process for the selected bike path.
- R15: When a logged-in user selects a previously automatically recorded bike path that has weather information, the BBP platform shall display the stored weather information for the selected bike path.
- R16: When a logged-in user selects a previously recorded bike path, the BBP platform shall display the list of streets traversed by the selected bike path in the stored segment order.

- R17: When a logged-in user selects a previously recorded bike path, the BBP platform shall display the obstacles associated with the selected bike path showing for each obstacle its type and the street where it was detected.

	Desktop Web App	Mobile Web App	Load Balancer	API Gateway
R8	X	X	X	X
R9	X	X	X	X
R10	X	X	X	X
R11	X	X	X	X
R12	X	X	X	X
R13	X	X	X	X
R14	X	X	X	X
R15	X	X	X	X
R16	X	X	X	X
R17	X	X	X	X

	Authentication Service	UsersDB	Paths Service	PathsDB	Search Service	SearchDB
R8	X	X	X	X		
R9	X	X	X	X		
R10	X	X	X	X		
R11	X	X	X	X	X	X
R12	X	X	X	X	X	X
R13	X	X	X	X		
R14	X	X	X	X		
R15	X	X	X	X		
R16	X	X	X	X		
R17	X	X	X	X		

	Automatic Path Recording Service	Message Broker	Statistics Service	Path Construction Service	Obstacle (Det.) Service
R8					
R9	X	X	X	X	
R10	X	X			
R11					
R12					
R13	X	X		X	X
R14	X	X	X		
R15	X	X			
R16	X	X		X	
R17	X	X		X	X

	Weather Service Provider	Web GPS Reciever	Mobile Sensor Manager	Maps Provider
R8				
R9			X	
R10	X			
R11				
R12				
R13				X
R14		X	X	
R15	X	X		
R16		X		
R17		X	X	

Searching of paths

- R18: When a user enters the BPP platform, it shall allow the user to search bike paths considering an origin and a destination point.
- R19: When a user performs a bike-path search and the BBP platform does not have recorded bike paths that satisfy the search parameters, the BBP platform shall generate candidate bike paths by combining recorded segments (including their associated obstacles) from existing bike paths.
- R20: The BBP platform shall display bike paths options (First one on a map), corresponding to an user search (with the origin and destination point) in descending order depending on their path score.

	Desktop Web App	Mobile Web App	Load Balancer	API Gateway
R18	X	X	X	X
R19	X	X	X	X
R20	X	X	X	X

	Authentication Service	UsersDB	Paths Service	PathsDB	Search Service	SearchDB
R18					X	X
R19			X	X	X	X
R20					X	X

	Weather Service Provider	Web GPS Reciever	Mobile Sensor Manager	Maps Provider
R18				X
R19				X
R20				X

Path information collection and merging

- R21: When the BBP platform receives new status and obstacle information from a public bike path, it shall update their path score according to all updates made to the path information, prioritizing the most up-to-date information.
- R22: When the BBP platform generates new bike paths in response to a user bike-path search, the BBP platform shall estimate and assign an overall status to each generated bike path, deriving that status from the statuses of the existing bike paths that include the segments used to form it.

	Desktop Web App	Mobile Web App	Load Balancer	API Gateway
R21	X	X	X	X
R22	X	X	X	X

	Authentication Service	UsersDB	Paths Service	PathsDB	Search Service	SearchDB
R21	X	X	X	X	X	X
R22			X	X	X	X

5 IMPLEMENTATION , INTEGRATION & TESTING

5.1 Overview

This section is dedicated to explaining the implementation, integration, and testing strategies to best verify the application’s functionalities.

5.2 Implementation Plan

The implementation plan of the BBP system follows a progressive, layered, and dependency-aware approach, ensuring that foundational components are available before higher-level services are developed. The order of implementation is aligned with the system’s microservices architecture, as seen in the C&C diagram, and supports incremental integration and testing.

The first part focuses on implementing persistent data storage, representing the foundation upon which all application services rely. It will include the PathDB, SearchDB, and UsersDB, since these components carry data information that is needed for the implementation of other components, such as path metadata, path scores, and user credentials.

After having the persistence available, it is possible to implement the respective services corresponding to the DBs, being the Paths Service, the Search Service, and the Authentication Service.

On the other hand, the following services operate asynchronously and process sensor data streams, and can be implemented disregarding the data storage components. They are Path Construction Service, Obstacle Detection Service, and Statistics Service.

From then, it is useful to follow the C&C diagram and implement the missing components in order of dependency. The Automatic Recording Service can be implemented, followed by all the External Services and the API Gateway. From this last one, the Load balancer is implemented. After that, to sum it up, Desktop Web App and Mobile Web App components can be implemented in the Presentation Layer.

5.3 Component Integration and Testing

The integration and testing of the BBP system follow a hybrid strategy combining Top-Down with Thread-based integration. This approach is chosen to ensure early validation of user-visible functionalities while progressively integrating lower-level components and infrastructure services in a controlled manner. It is also easier to keep track of, since no additional drivers are required, developing as a single graph. During early integration stages, Lower-level services, like Message Broker subscribers and external providers, are replaced with stubs. When components are implemented, stubs are respectively removed and replaced with real services. External services are mocked until internal component threads are stable, avoiding dependency on external availability.

The Top-Down integration starts from the Presentation Layer and API Gateway, progressively incorporating other services, as the Data Layer branches and the Path Recording Service component. After this, the Message Broker and its branches can be introduced. Finishing it off, the External Services can be incorporated, as previously said.

In parallel with the Top-Down approach, the thread-based integration develops in a way where each integration thread corresponds to a complete user-visible functionality spanning multiple components.

Thread 1 focuses on the Register and Log-in of users, validating the passing and checking of credentials that allow a user to register and log in to the BBP platform. It includes the Presentation Layer, API Gateway, Authentication Service, and User DB, inside the Data Layer, covering all the needed components for the validation.

Thread 2 focuses on the Manual Path recording, validating the complete flow from user input to persistent storage, ensuring that everything is correctly saved. It includes Presentation Layer, API Gateway, Authentication Service, Path Service, and PathDB, in the Data Layer, as components to assure the right integration.

Thread 3 focuses on the Automatic Path Recording, validating raw sensor data streaming, then integrating individual processing services one at a time, to finally test the aggregation and storage of the bike path. To fully achieve this, it is needed the inclusion of the Presentation Layer, API Gateway, Automatic Recording Service, Message Broker,

Path Construction Service, Obstacle Detection Service, Statistics Service, Path Service, and PathDB, inside the Data Layer.

Thread 4 focuses on the Path Search and Visualization, validating that users can search for paths, retrieve ranked results, and visualize routes correctly. It includes the Presentation Layer, API Gateway, Search Service, Path Service, and SearchDB and PathDB, both inside the Data Layer.

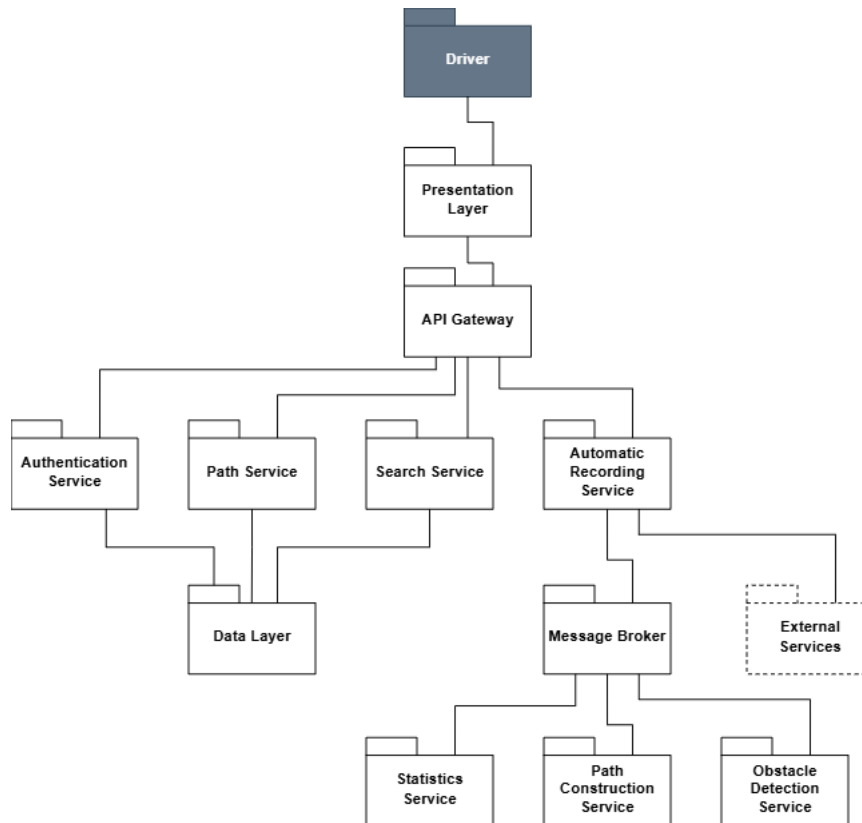


Figure 22: Finished version of the Integration Testing Diagram

5.4 System Testing

Once all BBP components have been fully integrated and verified through integration testing, it is possible to proceed to the System Testing as a whole, with the goal of validating that it satisfies all functional and non-functional requirements specified in the RASD document. System testing is conducted in an environment with real databases, deployed services, and integrated external providers where possible. Tests are executed from an end-user perspective, treating the system as a black box.

5.4.1 Functional System Testing

Functional system testing verifies that the BBP platform correctly implements the functional requirements and supports all documented use cases. The main functional scenarios tested include:

- User Log-in, register and authentication
- Manual bike path recording
- Automatic bike path recording, including sensor data streaming
- Management of paths, including visualization of statistics, obstacles and any suitable information
- Search, visualization and interaction of public bike paths

Each test case is easily related to the use cases and requirements, giving a better validation traceability.

5.4.2 Performance Testing

Performance testing evaluates the responsiveness and efficiency of the BBP platform, being coherent with the non-functional requirements mentioned in the RASD document. The testing objectives include:

- Ensuring safety of the system, regarding sensible information and data about users' locations, credentials and even external services communication
- Measuring response times for path search and visualization
- Assuring compatibility with multiple operating systems and devices
- Ensuring acceptable distance for location sensor data
- Ensuring the system keeps recording the path when the user chooses "Automatic Recording" even if the device loses connection temporarily
- Ensuring the API connecting internal and external system communication works correctly

The system's performance is tested under realistic usage patterns reflecting expected numbers of concurrent users and active recording sessions.

5.4.3 End-to-End Testing

All of the End-to-End testings validate complete user workflows across all layers of the system. It includes multiple layers and multiple scenarios, including anything a user is able to do in the platform, testing everything. In order to do this, it is also used the Load and Stress testing strategies. Load testing assesses the system's behavior under increasing workload levels, while stress testing evaluates system robustness beyond expected operational limits. They both aim to:

- Detect memory leaks or resource mismanagement
- Identify the maximum supported number of concurrent users and recording sessions
- Verify collapses under extreme conditions
- Ensure system recovery after overload or partial service failure

6 EFFORT SPENT

Hours worked	Section 1	Section 2	Section 3	Section 4	Section 5
Juan Sebastian Sarmiento Pulido	1	19	2	3	2
Pedro Augusto Villarinho dos Santos	1	4	2	1	14
Esteban Lopez Barreto	1	17	3	1	1

7 REFERENCES

1. The slides on the webeep page of the Software Engineering II course (Software architecture style, Microservices)
2. What is three-tier architecture?. IBM. <https://www.ibm.com/think/topics/three-tier-architecture>
3. What is Container orchestration. IBM. <https://www.ibm.com/think/topics/container-orchestration>
4. Reverse Proxy. IBM. <https://www.ibm.com/docs/en/aspera-fasp-proxy/1.4.x?topic=reverse-proxy>
5. What is auto scaling. IBM. <https://www.ibm.com/think/topics/autoscaling>
6. Course Notes: Software Architecture. Universidad Nacional de Colombia.